

UNIT III

DYNAMIC PROGRAMMING

Dynamic programming is a name coined by Richard Bellman in 1955. Dynamic programming, as a greedy method, is a powerful algorithm design technique that can be used when the solution to the problem may be viewed as the result of a sequence of decisions. In the greedy method we make irrevocable decisions one at a time, using a greedy criterion. However, in dynamic programming we examine the decision sequence to see whether an optimal decision sequence contains optimal decision subsequence.

When optimal decision sequences contain optimal decision subsequences, we can establish recurrence equations, called dynamic-programming recurrence equations that enable us to solve the problem in an efficient way.

Dynamic programming is based on the principle of optimality (also coined by Bellman). The principle of optimality states that no matter whatever the initial state and initial decision are, the remaining decision sequence must constitute an optimal decision sequence with regard to the state resulting from the first decision. The principle implies that an optimal decision sequence consists of optimal decision subsequences. Since the principle of optimality may not hold for some formulations of some problems, it is necessary to verify that it does hold for the problem being solved. Dynamic programming cannot be applied when this principle does not hold.

The steps in a dynamic programming solution are:

- Verify that the principle of optimality holds
- Set up the dynamic-programming recurrence equations
- Solve the dynamic-programming recurrence equations for the value of the optimal solution.
- Perform a trace back step in which the solution itself is constructed.

Dynamic programming differs from the greedy method since the greedy method produces only one feasible solution, which may or may not be optimal, while dynamic programming produces all possible sub-problems at most once, one of which guaranteed to be optimal. Optimal solutions to sub-problems are retained in a table, thereby avoiding the work of recomputing the answer every time a sub-problem is encountered.

The divide and conquer principle solve a large problem, by breaking it up into smaller problems which can be solved independently. In dynamic programming this principle is carried to an extreme: when we don't know exactly which smaller problems to solve, we simply solve them all, then store the answers away in a table to be used later in solving larger problems. Care is to be taken to avoid recomputing previously computed values; otherwise the recursive program will have prohibitive complexity. In some cases, the solution can be improved and in other cases, the dynamic programming technique is the best approach.

Two difficulties may arise in any application of dynamic programming:

1. It may not always be possible to combine the solutions of smaller problems to form the solution of a larger one.
2. The number of small problems to solve may be unacceptably large.

There is no precise definition of which problems can be effectively solved with dynamic programming; there are many hard problems for which it does not seem to be applicable, as well as many easy problems for which it is less efficient than standard algorithms.

OPTIMAL BINARY SEARCH TREES

A Binary Search Tree (BST) is a tree where the key values are stored in the internal nodes. The external nodes are null nodes. The keys are ordered lexicographically, i.e. for each internal node all the keys in the left subtree are less than the keys in the node, and all the keys in the right subtree are greater.

When we know the frequency of searching each one of the keys, it is quite easy to compute the expected cost of accessing each node in the tree. An optimal binary search tree is a BST, which has minimal expected cost of locating each node.

Search time of an element in a BST is $O(n)$, whereas in a Balanced-BST search time is $O(\log n)$. Again the search time can be improved in Optimal Cost Binary Search Tree, placing the most frequently used data in the root and closer to the root element, while placing the least frequently used data near leaves and in leaves.

Here, the Optimal Binary Search Tree Algorithm is presented. First, we build a BST from a set of provided n number of distinct keys $\langle k_1, k_2, k_3, \dots, k_n \rangle$. Here we assume, the probability of accessing a key K_i is p_i . Some dummy keys ($d_0, d_1, d_2, \dots, d_n$) are added as some searches may be performed for the values which are not present in the Key set K . We assume, for each dummy key d_i probability of access is q_i .

Algorithm OBST(p, q, n)

```
// Given  $n$  distinct identifiers  $a[1..n]$  and probabilities
//  $p[i], 1 \leq i \leq n$ , and  $q[i], 0 \leq i \leq n$ , this algorithm computes
// the cost  $c[i, j]$  of optimal binary search tree  $T$  for identifiers
//  $a[i+1], \dots, a[j]$ . It also computes  $r[i, j]$ , the root of  $T$ .
//  $w[i, j]$  is the weight of  $T$ .
{
    for  $i := 0$  to  $n$ 
    {
        // Initialize,
         $w[i, i] := q[i]; r[i, i] := 0; c[i, i] := 0.0;$ 
        // Optimal trees with one node
         $w[i+1, i+1] := q[i+1] + q[i] + p[i+1];$ 
         $r[i+1, i+1] := i+1;$ 
         $c[i+1, i+1] := q[i] + q[i+1] + p[i+1];$ 
    }
     $w[n, n] := q[n]; r[n, n] := 0; c[n, n] := 0.0;$ 
    for  $m := 2$  to  $n$  do // Find optimal trees with  $m$  nodes.
        for  $i := 0$  to  $n - m$  do
        {
             $j := i + m;$ 
             $w[i, j] := w[i, j-1] + p[j] + q[j]$ 

```

```

        k := Find(c, r, i, j);
        // A value of l in the ranger[i,j-1]<t
        //<=[r +1,j] that minimizes c[i,l-1]+c[l,j];
        c[i,j]:=w[i,j]+c[i,k-1]+c[k,j];
        r[i,j]:=k;
    }
    write (c[0,n], w[0,n],r[0,n]);
}
AlgorithmFind(c,r,i,j)
{
    min:=∞;
    for m :=r[i,j-1]to r[i+ 1,j]do
    if (c[i,m-1]+c[m, j])<min then
    {
        min:=c[i,m-1]+c[m,j];
        l:=m;
    }
    Return l;
}

```

Given a set of identifiers, we wish to create a binary search tree organization .We may expect different BST for the same identifiers to have different performance.

Let us assume that the given set of identifiers as $\{a_1, a_2, a_3, \dots, a_n\}$ with $a_1 < a_2 < a_3 < \dots < a_n$

Let $P(i)$ be the probability that we search for the identifier a_i .

Let $Q(i)$ be the probability that the identifier 'X' is being searched for 'l' ,such that $a \leq i \leq n$ i.e, $0 \leq i < n$

(Assume $a_0 = -\infty$ and $a_{n+1} = +\infty$)

$\sum_{0 \leq i \leq n} P(i)$ is the probability of successful search

$\sum_{0 \leq i \leq n} Q(i)$ is the probability of unsuccessful search

Then $\sum_{0 \leq i \leq n} P(i) + \sum_{0 \leq i \leq n} Q(i) = 1$

To obtain the cost function for BST ,it is useful to add a fictitious nodes in the place of every empty subtree in the search tree . such nodes are called **External nodes** and other nodes are called **Internal nodes**.

In the binary tree has n identifiers there will be exactly 'n; internal node and (n+1) external nodes

Hence, the expected cost contribution from the internal node for a_i is $p(i) * \text{level}(a_i)$.

The identifiers not in the binary search tree can be partitioned into $n + 1$ equivalence classes $E_i, 0 < i < n$

The class E_0 contains all identifiers x such that $x < a_1$.

The class E_i contains all identifiers x such that $a_i < x < a_{i+1}, 1 < i < n$.

The class E_n contains all identifiers $x, x > a_n$.

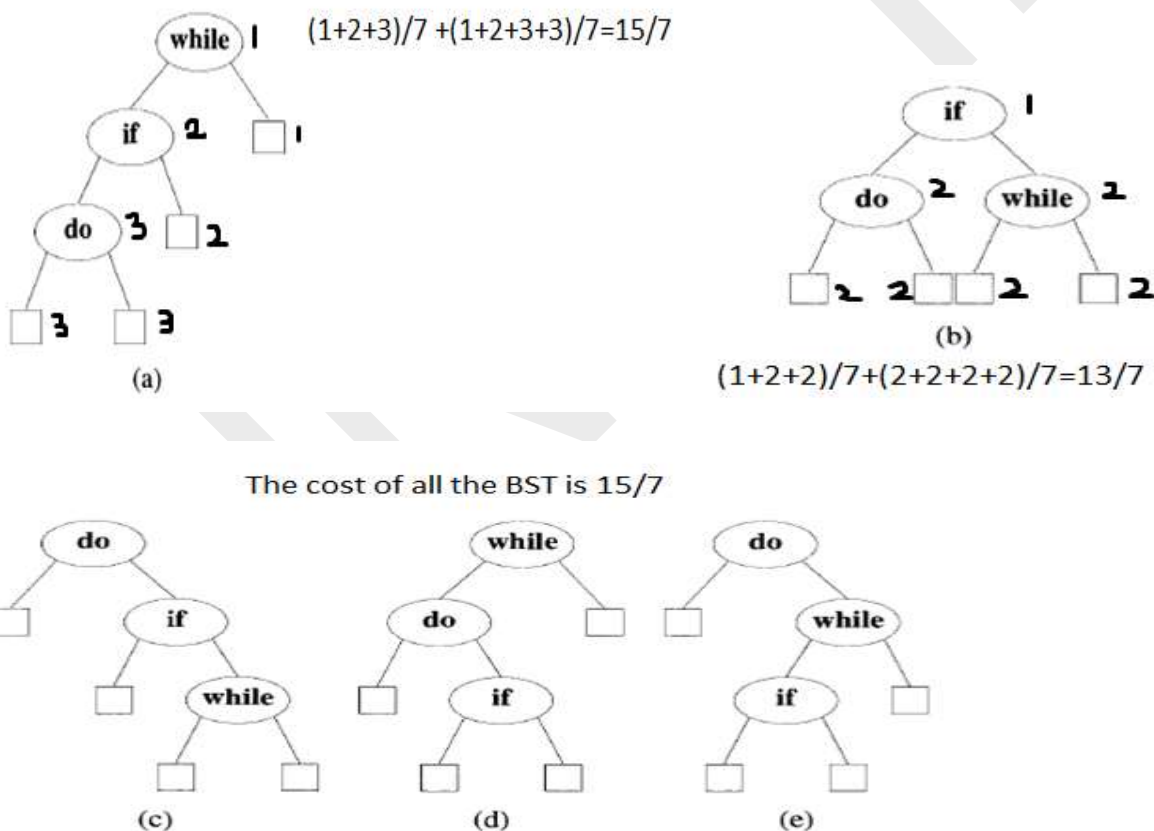
It is easy to see that for all identifiers in the same class E_i , the search terminates at the same external node. For identifiers in different E_i the search terminates at different external node

If the failure node for E_i is at level Z , then only 1 iteration of the while loop are made. Hence, the cost contribution of this node is $q(i) * (\text{level}(E_i) - 1)$.

The preceding discussion leads to the following formula for the expected cost of a binary search tree

$$\sum_{i \leq n \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q[i] * \text{level}(E_i) - 1$$

Example 1: The possible binary search trees for the identifier set $(a_1, a_2, a_3) = (\text{do}, \text{if}, \text{while})$ are given with equal probabilities $p(i) = q(i) = 1/7$ for all a_i ,



Example 2: The possible binary search trees for the identifier set $(a_1, a_2, a_3) = (\text{do}, \text{if}, \text{while})$ are given with equal probabilities With $p(1) = .5, p(2) = .1, p(3) = .05, q(0) = .15, q(1) = .1, q(2) = .05$ and $g(3) = .05$ we have

Here the contribution from successful searches is

$$\sum_{i \leq n} P(i) * level(ai)$$

$$3 * 0.5 + 2 * 0.1 + 0.05 = 1.75$$

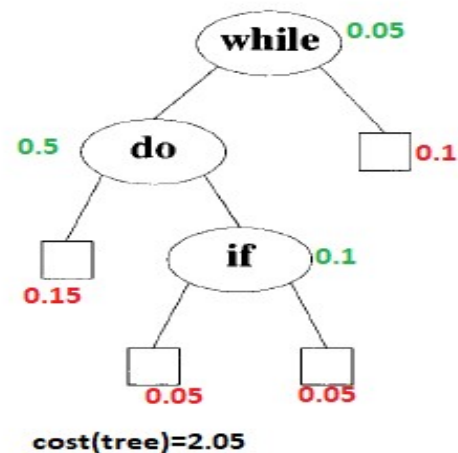
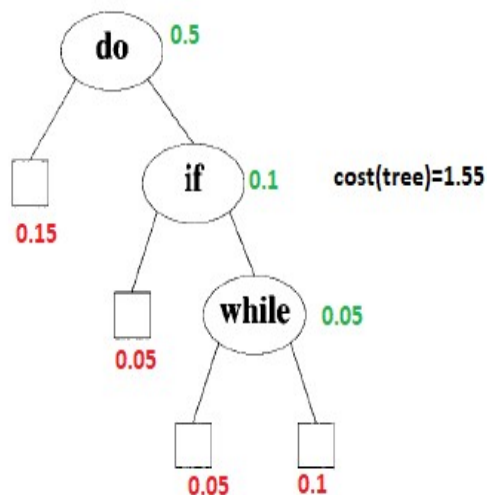
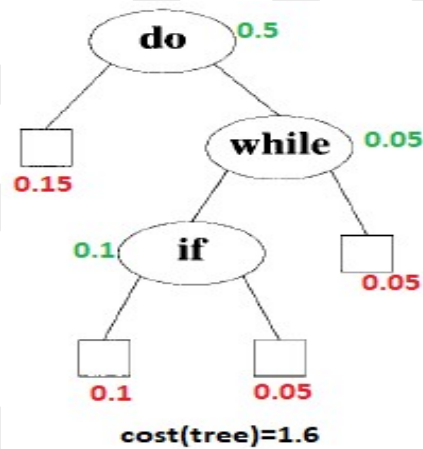
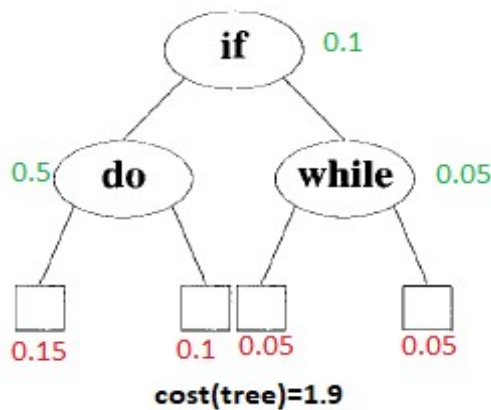
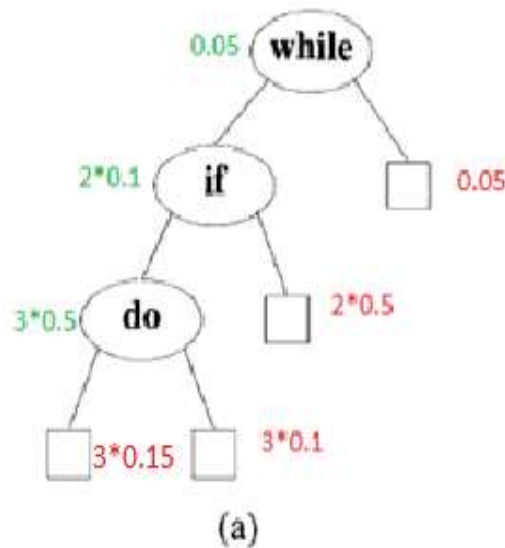
The contribution from unsuccessful searches is

$$\sum_{0 \leq i \leq n} Q[i] * level(Ei) - 1$$

$$3 * 0.15 + 3 * 0.1 + 2 * 0.05 + 0.05 = 0.90$$

$$\text{So total} = \sum_{i \leq n} P(i) * level(ai) + \sum_{0 \leq i \leq n} Q[i] * level(Ei) - 1$$

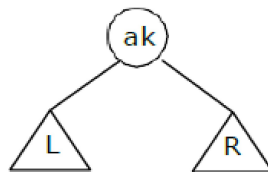
$$= (3 * 0.5 + 2 * 0.1 + 0.05) + (3 * 0.15 + 3 * 0.1 + 2 * 0.05 + 0.05) = 2.65$$



Out of all the TREES with the minimum cost is 1.55

Huffman coding tree solved by a greedy algorithm has a limitation of having the data only at the leaves and it must not preserve the property that all nodes to the left of the root have keys, which are less etc. Construction of an optimal binary search tree is harder, because the data is not constrained to appear only at the leaves, and also because the tree must satisfy the binary search tree property and it must preserve the property that all nodes to the left of the root have keys, which are less.

A dynamic programming solution to the problem of obtaining an optimal binary search tree can be viewed by constructing a tree as a result of sequence of decisions by holding the principle of optimality. A possible approach to this is to make a decision as to which of the a_i 's be arranged to the root node at 'T'. If we choose ' a_k ' then it is clear that the internal nodes for $a_1, a_2, \dots, a_{k-1}$ as well as the external nodes for the classes $E_0, E_1, \dots, E_{k-1}$ will lie in the left sub tree, L, of the root. The remaining nodes will be in the right subtree, R. The structure of an optimal binary search tree is:



$$\text{Cost (L)} = \sum_{i=1}^K P(i) * \text{level}(a_i) + \sum_{i=0}^K Q(i) * (\text{level}(E_i) - 1)$$

$$\text{Cost (R)} = \sum_{i=K}^n P(i) * \text{level}(a_i) + \sum_{i=K}^n Q(i) * (\text{level}(E_i) - 1)$$

The $C(i, J)$ can be computed as

$$C(i, J) = \min_{i < k < J} \{C(i, k-1) + C(k, J) + P(k) + w(i, k-1) + w(k, J)\}$$

$$= \min_{i < k < J} \{C(i, k-1) + C(k, J)\} + w(i, J) \text{----- (1)}$$

$$\text{Where } W(i, J) = P(J) + Q(J) + w(i, J-1) \text{----- (2)}$$

Initially $C(i, i) = 0$ and $w(i, i) = Q(i)$ for $0 < i < n$.

Equation (1) may be solved for $C(0, n)$ by first computing all $C(i, J)$ such that $J - i = 1$

Next, we can compute all $C(i, J)$ such that $J - i = 2$, Then all $C(i, J)$ with $J - i = 3$ and so on.

$C(i, J)$ is the cost of the optimal binary search tree 'Tij' during computation we record the root $R(i, J)$ of each tree 'Tij'. Then an optimal binary search tree may be constructed from these $R(i, J)$.

$R(i, J)$ is the value of ' K ' that minimizes equation (1).

We solve the problem by knowing $W(i, i+1)$, $C(i, i+1)$ and $R(i, i+1)$, $0 \leq i < 4$; Knowing $W(i, i+2)$, $C(i, i+2)$ and $R(i, i+2)$, $0 \leq i < 3$ and repeating until $W(0, n)$, $C(0, n)$ and $R(0, n)$ are obtained.

The results are tabulated to recover the actual tree.

Example: construct a OBST by using the following data $n = 4$, and $(a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$ Let $P(1:4) = (3, 3, 1, 1)$ and $Q(0:4) = (2, 3, 1, 1, 1)$

Solution: This computation is carried out row-wise from row 0 to row 4. Initially, $W(i, i) = Q(i)$ and $C(i, i) = 0$ and $R(i, i) = 0, 0 < i < 4$. Solving for $C(0, n)$:

By using the equations mentioned above 1 and 2 we solve the problem

We will consider the P and Q as

0	1	2	3	4
0	3	3	1	1

=P

An extra value is added for P

2	3	1	1	1
---	---	---	---	---

=Q

By considering $W(i, i) = Q(i)$ where $i=j$, the values of

$W(0,0)=Q(0)=2, W(1,1)=Q(1)=3, W(3,3)=Q(3)=1, W(4,4)=Q(4)=1$ and $W(5,5)=Q(5)=1$

Initially we need to consider $C(0,0)=C(1,1)=C(2,2)=C(3,3)=C(4,4)=0$

and $R(0,0)=R(1,1)=R(2,2)=R(3,3)=R(4,4)=0$

We need to calculate the values of $W(0,1)$ to $w(0,4)$ and $r(0,1)$ to $r(0,4)$ from this we get

	0	1	2	3	4
0	$W(0,0)=2$ $C(0,0)=0$ $R(0,0)=0$	$W(1,1)=2$ $C(1,1)=0$ $R(1,1)=0$	$W(2,2)=2$ $C(2,2)=0$ $R(2,2)=0$	$W(3,3)=2$ $C(3,3)=0$ $R(3,3)=0$	$W(4,4)=2$ $C(4,4)=0$ $R(4,4)=0$
1	$W(0,1)$ $C(0,1)$ $R(0,1)$	$W(1,2)$ $C(1,2)$ $R(1,2)$	$W(2,3)$ $C(2,3)$ $R(2,3)$	$W(3,4)$ $C(3,4)$ $R(3,4)$	
2	$W(0,2)$ $C(0,2)$ $R(0,2)$	$W(1,3)$ $C(1,3)$ $R(1,3)$	$W(2,4)$ $C(2,4)$ $R(2,4)$		
3	$W(0,3)$ $C(0,3)$ $R(0,3)$	$W(1,4)$ $C(1,4)$ $R(1,4)$			
4	$W(0,4)$ $C(0,4)$ $R(0,4)$				

First:

Computing all $C(i, j)$ such that $j - i = 1; j = i + 1$ and as $0 < i < 4; i = 0, 1, 2$ and $3; i < k \leq J$.

Start with $i = 0$; so $j = 1$; as $i < k \leq j$, so the possible value for $k = 1$

$$W(0, 1) = P(1) + Q(1) + W(0, 0) = 3 + 3 + 2 = 8$$

$$C(0, 1) = W(0, 1) + \min \{C(0, 0) + C(1, 1)\} = 8$$

$R(0, 1) = 1$ (the minimum value which we got from $c(0,1)$ i.e., $k=1$)

Next with $i = 1$; so $j = 2$; as $i < k \leq j$, so the possible value for $k = 2$

$$W(1, 2) = P(2) + Q(2) + W(1, 1) = 3 + 1 + 3 = 7$$

$$C(1, 2) = W(1, 2) + \min \{C(1, 1) + C(2, 2)\} = 7$$

$$R(1, 2) = 2$$

Next with $i = 2$; so $j = 3$; as $i < k \leq j$, so the possible value for $k = 3$

$$W(2, 3) = P(3) + Q(3) + W(2, 2) = 1 + 1 + 1 = 3$$

$$C(2, 3) = W(2, 3) + \min \{C(2, 2) + C(3, 3)\} = 3 + [(0 + 0)] = 3$$

$$R(2, 3) = 3$$

Next with $i = 3$; so $j = 4$; as $i < k \leq j$, so the possible value for $k = 4$

$$W(3, 4) = P(4) + Q(4) + W(3, 3) = 1 + 1 + 1 = 3$$

$$C(3, 4) = W(3, 4) + \min \{[C(3, 3) + C(4, 4)]\} = 3 + [(0 + 0)] = 3$$

$$R(3, 4) = 4$$

Second,

Computing all $C(i, j)$ such that $j - i = 2$; $j = i + 2$ and as $0 < i < 3$; $i = 0, 1, 2$; $i < k \leq J$.

Start with $i = 0$; so $j = 2$; as $i < k \leq J$, so the possible values for $k = 1$ and 2 .

$$W(0, 2) = P(2) + Q(2) + W(0, 1) = 3 + 1 + 8 = 12 \text{ fs}$$

$$\begin{aligned} C(0, 2) &= \mathbf{w(i,j)+\min\{c(0,1-1)+c(1,2),c(0,2-1)+c(2,2)\}} \text{ from equation 1} \\ &= W(0, 2) + \min \{(C(0, 0) + C(1, 2)), (C(0, 1) + C(2, 2))\} \\ &= 12 + \min \{(0 + 7, 8 + 0)\} = 19 \end{aligned}$$

$R(0, 2) = 1$ (the minimum value we got from the $k=1$ i.e., $C(0, 0) + C(1, 2)$)

Next, with $i = 1$; so $j = 3$; as $i < k \leq j$, so the possible value for $k = 2$ and 3 .

$$W(1, 3) = P(3) + Q(3) + W(1, 2) = 1 + 1 + 7 = 9$$

$$\begin{aligned} C(1, 3) &= \mathbf{w(i,j)+\min\{c(1,2-1)+c(2,3),c(1,3-1)+c(3,3)\}} \text{ from equation 1} \\ &= W(1, 3) + \min \{[C(1, 1) + C(2, 3)], [C(1, 2) + C(3, 3)]\} \\ &= W(1, 3) + \min \{(0 + 3), (7 + 0)\} = 9 + 3 = 12 \end{aligned}$$

$R(1, 3) = 2$ (the minimum value we got from the $k=2$ i.e., $C(1, 1) + C(2, 3)$)

Next, with $i = 2$; so $j = 4$; as $i < k \leq j$, so the possible value for $k = 3$ and 4 .

$$W(2, 4) = P(4) + Q(4) + W(2, 3) = 1 + 1 + 3 = 5$$

$$\begin{aligned} C(2, 4) &= W(2, 4) + \min \{[C(2, 2) + C(3, 4)], [C(2, 3) + C(4, 4)]\} \\ &= 5 + \min \{(0 + 3), (3 + 0)\} = 5 + 3 = 8 \end{aligned}$$

$R(2, 4) = 3$ (the minimum value we got from the $k=3$ i.e., $[C(2, 2) + C(3, 4)]$)

Third,

Computing all $C(i, j)$ such that $J - i = 3$; $j = i + 3$ and as $0 < i < 2$; $i = 0, 1$; $i < k \leq J$.

Start with $i = 0$; so $j = 3$; as $i < k \leq j$, so the possible values for $k = 1, 2$ and 3 .

$$W(0, 3) = P(3) + Q(3) + W(0, 2) = 1 + 1 + 12 = 14$$

$$C(0, 3) = \mathbf{w(i,j)+\min\{c(0,1-1)+c(1,3), c(0,2-1)+c(2,3),c(0,3-1)+c(3,3)\}} \text{ from equation 1}$$

$$=W(0,3) + \min \{ [C(0,0) + C(1,3)], [C(0,1) + C(2,3)], [C(0,2) + C(3,3)] \}$$

$$= 14 + \min \{ (0 + 12), (8 + 3), (19 + 0) \} = 14 + 11 = 25$$

$R(0,3) = 2$ (the minimum value we got from the $k=2$ i.e., $[C(0,1) + C(2,3)]$)

Start with $i = 1$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 2, 3$ and 4 .

$$W(1,4) = P(4) + Q(4) + W(1,3) = 1 + 1 + 9 = 11$$

$$C(1,4) = w(i,j) + \min \{ c(1,2-1) + c(2,4), c(1,3-1) + c(3,4), c(1,4-1) + c(4,4) \} \text{ from equation 1}$$

$$= W(1,4) + \min \{ [C(1,1) + C(2,4)], [C(1,2) + C(3,4)], [C(1,3) + C(4,4)] \}$$

$$= 11 + \min \{ (0 + 8), (7 + 3), (12 + 0) \} = 11 + 8 = 19$$

$$R(1,4) = 2$$

Fourth,

Computing all $C(i,j)$ such that $j - i = 4$; $j = i + 4$ and as $0 < i < 1$; $i = 0$; $i < k \leq j$.

Start with $i = 0$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3$ and 4 .

$$W(0,4) = P(4) + Q(4) + W(0,3) = 1 + 1 + 14 = 16$$

$$C(0,4) = w(i,j) + \min \{ c(0,1-1) + c(1,4), c(0,2-1) + c(2,4), c(0,3-1) + c(3,4), c(0,4-1) + c(4,4) \} \text{ from equation 1}$$

$$= W(0,4) + \min \{ [C(0,0) + C(1,4)], [C(0,1) + C(2,4)], [C(0,2) + C(3,4)], [C(0,3) + C(4,4)] \}$$

$$= 16 + \min \{ 0 + 19, 8 + 8, 19 + 3, 25 + 0 \}$$

$$= 16 + 16 = 32$$

$R(0,4) = 2$ (the minimum is generated when $k=2$)

	0	1	2	3	4
0	W(0,0)=2 C(0,0)=0 R(0,0)=0	W(1,1)=2 C(1,1)=0 R(1,1)=0	W(2,2)=2 C(2,2)=0 R(2,2)=0	W(3,3)=2 C(3,3)=0 R(3,3)=0	W(4,4)=2 C(4,4)=0 R(4,4)=0
1	W(0,1)=8 C(0,1)=8 R(0,1)=1	W(1,2)=7 C(1,2)=7 R(1,2)=2	W(2,3)=3 C(2,3)=3 R(2,3)=3	W(3,4)=3 C(3,4)=3 R(3,4)=4	
2	W(0,2)=12 C(0,2)=19 R(0,2)=1	W(1,3)=9 C(1,3)=12 R(1,3)=2	W(2,4)=5 C(2,4)=8 R(2,4)=3		
3	W(0,3)=14 C(0,3)=25 R(0,3)=2	W(1,4)=11 C(1,4)=19 R(1,4)=2			
4	W(0,4)=16 C(0,4)=32 R(0,4)=2				

From the table we see that $C(0,4) = 32$ is the minimum cost of a binary search tree for (a_1, a_2, a_3, a_4) . The root of the tree 'T04' is 'a2'.

Hence the left sub tree is 'T01' and right sub tree is T24.

root of 'T01' is 'a1' and the root of 'T24' is a3.

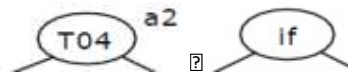
The left and right sub trees for 'T01' are 'T00' and 'T11' respectively. The root of T01 is 'a1'

The left and right sub trees for T24 are T22 and T34 respectively. The root of T24 is 'a3'.

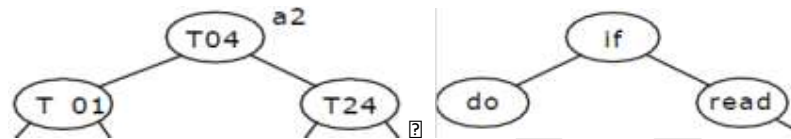
The root of T22 is null the root of T34 is a4.

Or

The minimum is generated from $R(0,4)$, whose value is 2^{nd} key then root node is "if "

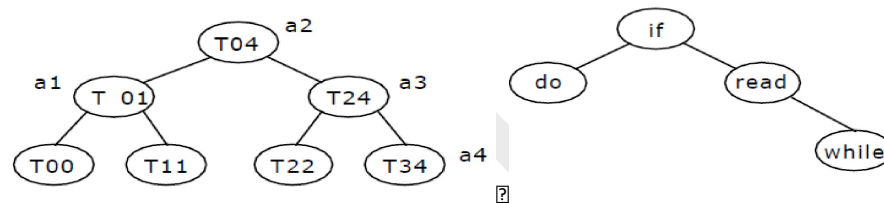


When the root node is 2^{nd} key, the left side node value will be $R(0,1)$ $k=1$ and right side node value will be $R(2,4)$ $k=3$



$R(2,4)$ the value that is left is $R(3,4)$ $k=4$ which is right node for the $r(2,4)$

The remaining nodes are the null nodes which are added to every internal nodes



Example 3:

Consider four elements a_1, a_2, a_3 and a_4 with $Q_0 = 1/8, Q_1 = 3/16, Q_2 = Q_3 = Q_4 = 1/16$ and $p_1 = 1/4, p_2 = 1/8, p_3 = p_4 = 1/16$. Construct an optimal binary search tree. Solving for $C(0, n)$:

First

Computing all $C(i, j)$ such that $j - i = 1; j = i + 1$ and as $0 < i < 4; i = 0, 1, 2$ and $3; i < k \leq j$.

Start with $i = 0$; so $j = 1$; as $i < k \leq j$, so the possible value for $k = 1$

$$W(0, 1) = P(1) + Q(1) + W(0, 0) = 4 + 3 + 2 = 9$$

$$C(0, 1) = W(0, 1) + \min \{C(0, 0) + C(1, 1)\} = 9 + [(0 + 0)] = 9$$

$$R(0, 1) = 1$$

(Value of 'K' that is minimum in the above equation). Next with $i = 1$; so $j = 2$; as $i < k \leq j$, so the possible value for $k = 2$

$$W(1, 2) = P(2) + Q(2) + W(1, 1) = 2 + 1 + 3 = 6$$

$$C(1, 2) = W(1, 2) + \min \{C(1, 1) + C(2, 2)\} = 6 + [(0 + 0)] = 6$$

$$R(1, 2) = 2$$

Next with $i = 2$; so $j = 3$; as $i < k \leq j$, so the possible value for $k = 3$

$$W(2, 3) = P(3) + Q(3) + W(2, 2) = 1 + 1 + 1 = 3$$

$$C(2, 3) = W(2, 3) + \min \{C(2, 2) + C(3, 3)\} = 3 + [(0 + 0)] = 3$$

$$R(2, 3) = 3$$

Next with $i = 3$; so $j = 4$; as $i < k \leq j$, so the possible value for $k = 4$

$$W(3, 4) = P(4) + Q(4) + W(3, 3) = 1 + 1 + 1 = 3$$

$$C(3, 4) = W(3, 4) + \min \{[C(3, 3) + C(4, 4)]\} = 3 + [(0 + 0)] = 3$$

$$R(3, 4) = 4$$

Second

Computing all $C(i, j)$ such that $j - i = 2$; $j = i + 2$ and as $0 < i < 3$; $i = 0, 1, 2$; $i < k \leq j$

Start with $i = 0$; so $j = 2$; as $i < k \leq j$,

so the possible values for $k = 1$ and 2 .

$$W(0, 2) = P(2) + Q(2) + W(0, 1) = 2 + 1 + 9 = 12$$

$$C(0, 2) = W(0, 2) + \min \{[C(0, 0) + C(1, 2)], [C(0, 1) + C(2, 2)]\}$$

$$= 12 + \min \{(0 + 6, 9 + 0)\} = 12 + 6 = 18$$

$$R(0, 2) = 1$$

Next, with $i = 1$; so $j = 3$; as $i < k \leq j$, so the possible value for $k = 2$ and 3 .

$$W(1, 3) = P(3) + Q(3) + W(1, 2) = 1 + 1 + 6 = 8$$

$$C(1, 3) = W(1, 3) + \min \{[C(1, 1) + C(2, 3)], [C(1, 2) + C(3, 3)]\}$$

$$= W(1, 3) + \min \{(0 + 3), (6 + 0)\} = 8 + 3 = 11$$

$$R(1, 3) = 2$$

Next, with $i = 2$; so $j = 4$; as $i < k \leq j$, so the possible value for $k = 3$ and 4 .

$$W(2, 4) = P(4) + Q(4) + W(2, 3) = 1 + 1 + 3 = 5$$

$$C(2, 4) = W(2, 4) + \min \{[C(2, 2) + C(3, 4)], [C(2, 3) + C(4, 4)]\}$$

$$= 5 + \min \{(0 + 3), (3 + 0)\} = 5 + 3 = 8$$

$$R(2, 4) = 3$$

Third,

Computing all $C(i, j)$ such that $J - i = 3$; $j = i + 3$ and as $0 < i < 2$; $i = 0, 1$;

$i < k \leq J$. Start with $i = 0$; so $j = 3$; as $i < k \leq j$, so the possible values for $k = 1, 2$ and 3 .

$$W(0, 3) = P(3) + Q(3) + W(0, 2) = 1 + 1 + 12 = 14$$

$$C(0, 3) = W(0, 3) + \min \{[C(0, 0) + C(1, 3)], [C(0, 1) + C(2, 3)], [C(0, 2) + C(3, 3)]\}$$

$$= 14 + \min \{(0 + 11), (9 + 3), (18 + 0)\} = 14 + 11 = 25$$

$$R(0, 3) = 1$$

Start with $i = 1$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 2, 3$ and 4 .

$$W(1, 4) = P(4) + Q(4) + W(1, 3) = 1 + 1 + 8 = 10$$

$$C(1, 4) = W(1, 4) + \min \{[C(1, 1) + C(2, 4)], [C(1, 2) + C(3, 4)], [C(1, 3) + C(4, 4)]\}$$

$$= 10 + \min \{(0 + 8), (6 + 3), (11 + 0)\} = 10 + 8 = 18$$

$$R(1, 4) = 2$$

Fourth,

Computing all $C(i, j)$ such that $J - i = 4$; $j = i + 4$ and as $0 < i < 1$; $i = 0$;

$i < k \leq J$. Start with $i = 0$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3$ and 4 .

$$W(0, 4) = P(4) + Q(4) + W(0, 3) = 1 + 1 + 14 = 16$$

$$C(0, 4) = W(0, 4) + \min \{[C(0, 0) + C(1, 4)], [C(0, 1) + C(2, 4)], [C(0, 2) + C(3, 4)], [C(0, 3) + C(4, 4)]\}$$

$$= 16 + \min [0 + 18, 9 + 8, 18 + 3, 25 + 0] = 16 + 17 = 33$$

$$R(0, 4) = 2$$

Table for recording $W(i, j)$, $C(i, j)$ and $R(i, j)$

Column Row	0	1	2	3	4
0	2, 0, 0	1, 0, 0	1, 0, 0	1, 0, 0,	1, 0, 0
1	9, 9, 1	6, 6, 2	3, 3, 3	3, 3, 4	
2	12, 18, 1	8, 11, 2	5, 8, 3		
3	14, 25, 2	11, 18, 2			
4	16, 33, 2				

From the table we see that $C(0, 4) = 33$ is the minimum cost of a binary search tree for (a_1, a_2, a_3, a_4)

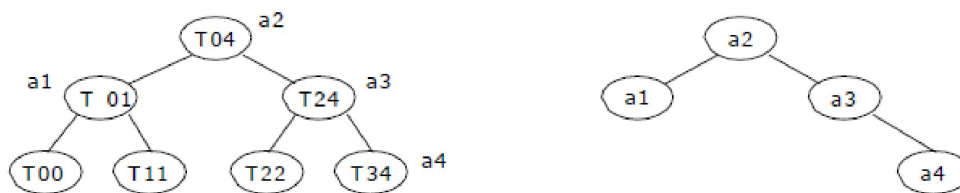
The root of the tree 'T04' is 'a2'.

Hence the left sub tree is 'T01' and the right sub tree is T24. The root of 'T01' is 'a1' and the root of 'T24' is a3.

The left and right subtrees for 'T01' are 'T00' and 'T11' respectively. The root of T01 is 'a1'

The left and right subtrees for T24 are T22 and T34 respectively. The root of T24 is 'a3'.

The root of T22 is null. The root of T34 is a4.



Example 4: given words are (A, B, C, D, E, F, G) with the probability $(4, 2, 1, 3, 5, 2, 1)$ and all other elements have zero probability.

Solving $c(0, n)$:

First computing all $c(i, j)$ such that $j - i = 1$; $j = i + 1$ and as $0 \leq i < 7$; $i = 0, 1, 2, 3, 4, 5$ and 6 ; $i < k \leq j$. Start with $i = 0$; so $j = 1$; as $i < k \leq j$, so the possible value for $k = 1$

$$W(0, 1) = P(1) + Q(1) + W(0, 0) = 4 + 0 + 0 = 4$$

$$C(0, 1) = W(0, 1) + \min \{C(0, 0) + C(1, 1)\} = 4 + [(0 + 0)] = 4$$

$$R(0, 1) = 1$$

Next with $i = 1$; so $j = 2$; as $i < k \leq j$, so the possible value for $k = 2$

$$W(1, 2) = P(2) + Q(2) + W(1, 1) = 2 + 0 + 0 = 2$$

$$C(1, 2) = W(1, 2) + \min \{C(1, 1) + C(2, 2)\} = 2 + [(0 + 0)] = 2$$

$$R(1, 2) = 2$$

Next with $i = 2$; so $j = 3$; as $i < k \leq j$, so the possible value for $k = 3$

$$W(2, 3) = P(3) + Q(3) + W(2, 2) = 1 + 0 + 0 = 1$$

$$C(2, 3) = W(2, 3) + \min \{C(2, 2) + C(3, 3)\} = 1 + [(0 + 0)] = 1$$

$$R(2, 3) = 3$$

Next with $i = 3$; so $j = 4$; as $i < k \leq j$, so the possible value for $k = 4$

$$W(3, 4) = P(4) + Q(4) + W(3, 3) = 3 + 0 + 0 = 3$$

$$C(3, 4) = W(3, 4) + \min \{C(3, 4) + C(4, 4)\} = 3 + [(0 + 0)] = 3$$

$$R(3, 4) = 4$$

Next with $i = 4$; so $j = 5$; as $i < k \leq j$, so the possible value for $k = 5$

$$W(4, 5) = P(5) + Q(5) + W(4, 4) = 5 + 0 + 0 = 5$$

$$C(4, 5) = W(4, 5) + \min \{C(4, 4) + C(5, 5)\} = 5 + [(0 + 0)] = 5$$

$$R(4, 5) = 5$$

Next with $i = 5$; so $j = 6$; as $i < k \leq j$, so the possible value for $k = 6$

$$W(5, 6) = P(6) + Q(6) + W(5, 5) = 2 + 0 + 0 = 2$$

$$C(5, 6) = W(5, 6) + \min \{C(5, 5) + C(6, 6)\} = 2 + [(0 + 0)] = 2$$

$$R(5, 6) = 6$$

Next with $i = 6$; so $j = 7$; as $i < k \leq j$, so the possible value for $k = 7$

$$W(6, 7) = P(7) + Q(7) + W(6, 6) = 1 + 0 + 0 = 1$$

$$C(6, 7) = W(6, 7) + \min \{C(6, 6) + C(7, 7)\} = 1 + [(0 + 0)] = 1$$

$$R(6, 7) = 7$$

Second,

computing all $c(i, j)$ such that $j - i = 2$; $j = i + 2$ and as $0 \leq i < 6$; $i = 0, 1, 2, 3, 4$ and 5 ; $i < k \leq j$; Start with $i = 0$; so $j = 2$; as $i < k \leq j$, so the possible values for $k = 1$ and 2 .

$$W(0, 2) = P(2) + Q(2) + W(0, 1) = 2 + 0 + 4 = 6$$

$$C(0, 2) = W(0, 2) + \min \{C(0, 0) + C(1, 2), C(0, 1) + C(2, 2)\}$$

$$= 6 + \min \{0 + 2, 4 + 0\} = 8$$

$$R(0, 2) = 1$$

Next with $i = 1$; so $j = 3$; as $i < k \leq j$, so the possible values for $k = 2$ and 3 .

$$W(1, 3) = P(3) + Q(3) + W(1, 2) = 1 + 0 + 2 = 3$$

$$C(1, 3) = W(1, 3) + \min \{C(1, 1) + C(2, 3), C(1, 2) + C(3, 3)\}$$

$$= 3 + \min \{0 + 1, 2 + 0\} = 4$$

$$R(1, 3) = 2$$

Next with $i = 2$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 3$ and 4 .

$$W(2, 4) = P(4) + Q(4) + W(2, 3) = 3 + 0 + 1 = 4$$

$$C(2, 4) = W(2, 4) + \min \{C(2, 2) + C(3, 4), C(2, 3) + C(4, 4)\}$$

$$= 4 + \min \{0 + 3, 1 + 0\} = 5$$

$$R(2, 4) = 4$$

Next with $i = 3$; so $j = 5$; as $i < k \leq j$, so the possible values for $k = 4$ and 5 .

$$W(3, 5) = P(5) + Q(5) + W(3, 4) = 5 + 0 + 3 = 8$$

$$C(3, 5) = W(3, 5) + \min \{C(3, 3) + C(4, 5), C(3, 4) + C(5, 5)\}$$

$$= 8 + \min \{0 + 5, 3 + 0\} = 11$$

$$R(3, 5) = 5$$

Next with $i = 4$; so $j = 6$; as $i < k \leq j$, so the possible values for $k = 5$ and 6 .

$$W(4, 6) = P(6) + Q(6) + W(4, 5) = 2 + 0 + 5 = 7$$

$$C(4, 6) = W(4, 6) + \min \{C(4, 4) + C(5, 6), C(4, 5) + C(6, 6)\} \\ = 7 + \min\{0 + 2, 5 + 0\} = 9$$

$$R(4, 6) = 5$$

Next with $i = 5$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 6$ and 7 .

$$W(5, 7) = P(7) + Q(7) + W(5, 6) = 1 + 0 + 2 = 3$$

$$C(5, 7) = W(5, 7) + \min \{C(5, 5) + C(6, 7), C(5, 6) + C(7, 7)\} \\ = 3 + \min\{0 + 1, 2 + 0\} = 4$$

$$R(5, 7) = 6$$

Third, computing all $c(i, j)$ such that $j - i = 3$; $j = i + 3$ and as $0 \leq i < 5$; $i = 0, 1, 2, 3, 4$ and $i < k \leq j$.

Start with $i = 0$; so $j = 3$; as $i < k \leq j$, so the possible values for $k = 1, 2$ and 3 .

$$W(0, 3) = P(3) + Q(3) + W(0, 2) = 1 + 0 + 6 = 7$$

$$C(0, 3) = W(0, 3) + \min \{C(0, 0) + C(1, 3), C(0, 1) + C(2, 3), C(0, 2) + C(3, 3)\} \\ = 7 + \min\{0 + 4, 4 + 1, 8 + 0\} = 7$$

$$R(0, 3) = 1$$

Next with $i = 1$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 2, 3$ and 4 .

$$W(1, 4) = P(4) + Q(4) + W(1, 3) = 3 + 0 + 3 = 6$$

$$C(1, 4) = W(1, 4) + \min \{C(1, 1) + C(2, 4), C(1, 2) + C(3, 4), C(1, 3) + C(4, 4)\} \\ = 6 + \min\{0 + 5, 2 + 3, 4 + 0\} = 10$$

$$R(1, 4) = 4$$

Next with $i = 2$; so $j = 5$; as $i < k \leq j$, so the possible values for $k = 3, 4$ and 5 .

$$W(2, 5) = P(5) + Q(5) + W(2, 4) = 5 + 0 + 4 = 9$$

$$C(2, 5) = W(2, 5) + \min \{C(2, 2) + C(3, 5), C(2, 3) + C(4, 5), C(2, 4) + C(5, 5)\} \\ = 9 + \min\{0 + 11, 1 + 5, 5 + 0\} = 14$$

$$R(2, 5) = 5$$

Next with $i = 3$; so $j = 6$; as $i < k \leq j$, so the possible values for $k = 4, 5$ and 6 .

$$W(3, 6) = P(6) + Q(6) + W(3, 5) = 2 + 0 + 8 = 10$$

$$C(3, 6) = W(3, 6) + \min \{C(3, 3) + C(4, 6), C(3, 4) + C(5, 6), C(3, 5) + C(6, 6)\} \\ = 10 + \min\{0 + 9, 3 + 2, 11 + 0\} = 15$$

$$R(3, 6) = 5$$

Next with $i = 4$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 5, 6$ and 7 .

$$W(4, 7) = P(7) + Q(7) + W(4, 6) = 1 + 0 + 7 = 8$$

$$C(4, 7) = W(4, 7) + \min \{C(4, 4) + C(5, 7), C(4, 5) + C(6, 7), C(4, 6) + C(7, 7)\} \\ = 8 + \min\{0 + 4, 5 + 1, 9 + 0\} = 12$$

$$R(4, 7) = 5$$

Fourth,

computing all $c(i, j)$ such that $j - i = 4$; $j = i + 4$ and as $0 \leq i < 4$; $i = 0, 1, 2, 3$ for $i < k \leq j$.

Start with $i = 0$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3$ and 4 .

$$W(0, 4) = P(4) + Q(4) + W(0, 3) = 3 + 0 + 7 = 10$$

$$C(0, 4) = W(0, 4) + \min \{C(0, 0) + C(1, 4), C(0, 1) + C(2, 4), C(0, 2) + C(3, 4), C(0, 3) + C(4, 4)\}$$

$$= 10 + \min\{0 + 10, 4 + 5, 8 + 3, 11 + 0\} = 19$$

$$R(0, 4) = 2$$

Next with $i = 1$; so $j = 5$; as $i < k \leq j$, so the possible values for $k = 2, 3, 4$ and 5 .

$$W(1, 5) = P(5) + Q(5) + W(1, 4) = 5 + 0 + 6 = 11$$

$$C(1, 5) = W(1, 5) + \min \{C(1, 1) + C(2, 5), C(1, 2) + C(3, 5), C(1, 3) + C(4, 5), C(1, 4) + C(5, 5)\}$$

$$= 11 + \min\{0 + 14, 2 + 11, 4 + 5, 10 + 0\} = 20$$

$$R(1, 5) = 4$$

Next with $i = 2$; so $j = 6$; as $i < k \leq j$, so the possible values for $k = 3, 4, 5$ and 6 .

$$W(2, 6) = P(6) + Q(6) + W(2, 5) = 2 + 0 + 9 = 11$$

$$C(2, 6) = W(2, 6) + \min \{C(2, 2) + C(3, 6), C(2, 3) + C(4, 6), C(2, 4) + C(5, 6), C(2, 5) + C(6, 6)\}$$

$$= 11 + \min\{0 + 15, 1 + 9, 5 + 2, 14 + 0\} = 18$$

$$R(2, 6) = 5$$

Next with $i = 3$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 4, 5, 6$ and 7 .

$$W(3, 7) = P(7) + Q(7) + W(3, 6) = 1 + 0 + 11 = 12$$

$$C(3, 7) = W(3, 7) + \min \{C(3, 3) + C(4, 7), C(3, 4) + C(5, 7), C(3, 5) + C(6, 7), C(3, 6) + C(7, 7)\}$$

$$= 12 + \min\{0 + 12, 3 + 4, 11 + 1, 15 + 0\} = 19$$

$$R(3, 7) = 5$$

Fifth,

computing all $c(i, j)$ such that $j - i = 5$; $j = i + 5$ and as $0 \leq i < 3$;

$i = 0, 1, 2$, $i < k \leq j$. Start with $i = 0$; so $j = 4$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3, 4$ and 5 .

$$W(0, 5) = P(5) + Q(5) + W(0, 4) = 5 + 0 + 10 = 15$$

$$C(0, 5) = W(0, 5) + \min \{C(0, 0) + C(1, 5), C(0, 1) + C(2, 5), C(0, 2) + C(3, 5), C(0, 3) + C(4, 5), C(0, 4) + C(5, 5)\}$$

$$= 10 + \min\{0 + 20, 4 + 14, 8 + 11, 19 + 0\} = 28$$

$$R(0, 5) = 2$$

Next with $i = 1$; so $j = 6$; as $i < k \leq j$, so the possible values for $k = 2, 3, 4, 5$ & 6 .

$$W(1, 6) = P(6) + Q(6) + W(1, 5) = 2 + 0 + 11 = 13$$

$$C(1, 6) = W(1, 6) + \min \{C(1, 1) + C(2, 6), C(1, 2) + C(3, 6), C(1, 3) + C(4, 6), C(1, 4) + C(5, 6), C(1, 5) + C(6, 6)\}$$

$$= 13 + \min\{0 + 18, 2 + 15, 4 + 9, 10 + 2, 20 + 0\} = 25$$

$$R(1, 6) = 5$$

Next with $i = 2$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 3, 4, 5, 6$ and 7 .

$$W(2, 7) = P(7) + Q(7) + W(2, 6) = 1 + 0 + 11 = 12$$

$$C(2, 7) = W(2, 7) + \min \{C(2, 2) + C(3, 7), C(2, 3) + C(4, 7), C(2, 4) + C(5, 7), C(2, 5) + C(6, 7), C(2, 6) + C(7, 7)\}$$

$$= 12 + \min\{0 + 18, 1 + 12, 5 + 4, 14 + 1, 18 + 0\} = 21$$

$$R(2, 7) = 5$$

Sixth,

computing all $c(i, j)$ such that $j - i = 6$; $j = i + 6$ and as $0 \leq i < 2$; $i = 0, 1$ $i < k \leq j$.

Start with $i = 0$; so $j = 6$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3, 4, 5$ & 6 .

$$W(0, 6) = P(6) + Q(6) + W(0, 5) = 2 + 0 + 15 = 17$$

$$C(0,6)=W(0,6)+\min\{C(0,0)+C(1,6),C(0,1)+C(2,6),C(0,2)+C(3,6),C(0,3)+C(4,6),C(0,4)+C(5,6),C(0,5)+C(6,6)\}$$

$$= 17 + \min\{0 + 25, 4 + 18, 8 + 15, 19 + 2, 31 + 0\} = 37$$

$$R(0, 6) = 4$$

Next with $i = 1$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 2, 3, 4, 5, 6$ and 7 .

$$W(1, 7) = P(7) + Q(7) + W(1, 6) = 1 + 0 + 13 = 14$$

$$C(1,7)=W(1,7)+\min\{C(1,1)+C(2,7),C(1,2)+C(3,7),C(1,3)+C(4,7),C(1,4)+C(5,7),C(1,5)+C(6,7),C(1,6)+C(7,7)\}$$

$$= 14 + \min\{0 + 21, 2 + 18, 4 + 12, 10 + 4, 20 + 1, 21 + 0\} = 28$$

$$R(1, 7) = 5$$

Seventh,

computing all $c(i, j)$ such that $j - i = 7$; $j = i + 7$ and as $0 \leq i < 1$; $i = 0$; $0 < k \leq j$. Start with $i = 0$; so $j = 7$; as $i < k \leq j$, so the possible values for $k = 1, 2, 3, 4, 5, 6$ and 7 .

$$W(0, 7) = P(7) + Q(7) + W(0, 6) = 1 + 0 + 17 = 18$$

$$C(0,7)=W(0,7)+\min\{C(0,0)+C(1,7),C(0,1)+C(2,7),C(0,2)+C(3,7),C(0,3)+C(4,7),C(0,4)+C(5,6),C(0,5)+C(6,7),C(0,6)+C(7,7)\}$$

$$= 18 + \min\{0 + 28, 4 + 21, 8 + 18, 19 + 4, 31 + 1, 37 + 0\} = 41$$

$$R(0, 7) = 4$$

The algorithm requires $O(n^3)$ time, since three nested for loops are used. Each of these loops takes on at most n values.

0/1 KNAPSACK PROBLEM

We are given a 'n' objects & knapsack with the capacity 'w', in which the object 'i' has the weight 'wi' is to be placed. Then the profit can be carried is 'PiXi'. The Object is to obtain the capacity of Knapsack to be filled with 'W' and to earn maximum profit.

The formula for this problem is

$$\text{Max} \sum_{i=1}^n P_i X_i \text{ subject to Constraints, where } \sum_{i=1}^n W_i X_i \leq W$$

Where $i \leq i \leq n$ at $X_i = 0$ or 1 and 'n' is the total number of objects.

Here we are going to consider the objects with highest weight, i.e., the value is either '1' (value is considered) or '0' (value is not considered)

The Dynamic solution for the 0/1 knapsack is

$$f_n(W) = \text{Max} \{f_{n-1}(W), f_{n-1}(W - W_n) + P_n\} \text{-----} \textcircled{1}$$

$$f_{n-1}(W) \rightarrow X_n = 1$$

$$f_{n-1}(W - W_n) \rightarrow X_n = 0$$

For arbitrary $F_i(y)$, $i > 0$ in above equation 1 can be generalized to

$$F_i(y) = \text{Max} \{f_{i-1}(y), f_{i-1}(y - W_i) + P_i\} \text{-----} \textcircled{2}$$

Equation 2 can be solved for $f_n(m)$ by beginning with the knowledge $f_0(y) = 0$ for all y and $f_i(y) = -\infty$, $y < 0$. Then $f_1, f_2, \dots, f_n$, can be successively computed using equation 2

So, we need to compute only $f_i(y_j)$, $1 \leq j \leq k$. We use the ordered set $S_i = \{(f(y_j), y_j) \mid i \leq j \leq k\}$ to represent $f_i(y)$. Each member of S_i is a pair (P, W) , where $P = f_i(y_i)$ and $W = y_i$

Notice that $S_0 = \{(0, 0)\}$.

We can compute S_{i+1} from S_i by first computing

$$S_{i1} = \{(P, W) \mid (P - P_i, W - W_i) \in S_1\}$$

S_{i+1} can be obtained by merging the pairs of S_i and S_{i1}

S_{i+1} contains 2 pairs (P_j, W_j) & (P_k, W_k) with the property that $P_j \leq P_k$ and $W_j > W_k$ then the pair (P_j, W_j) can be discarded because of equation 2. Discarding or purging rules such as this one are also known as **dominance rules**

Algorithm

```

For w = 0 to W
    B[0,w] = 0
for i = 1 to n
    B[i,0] = 0
for i = 1 to n
    for w = 0 to W
        if  $w_i \leq w$  // item i can be part of the solution
            if  $b_i + B[i-1, w-w_i] > B[i-1, w]$ 
                 $B[i, w] = b_i + B[i-1, w-w_i]$ 
            else
                 $B[i, w] = B[i-1, w]$ 
        else
             $B[i, w] = B[i-1, w]$  //  $w_i > w$ 

```

Algorithm

```

Algorithm DKnap(p,w,x,n,to)
{
    // pair[] is an array of PWs. b[0]:=1; pair[l].p:=pair[l].w:=0.0; //  $S^0$ 
    t:=l; h:=1; // Start and end of  $S^0$ 
    b[l] := next:=2; // Next free spot in pair[]
    for i := 1 to n do
        {
            // Generate  $S^1$ 
            k := t;
            u := Largest(pair, w, t, h, i, m);
            for j := t to u do
                {
                    // Generate  $S^i$  and merge.
                    pp := pair[j].p + p[i];
                    ww := pair[j].w + w[i];
                    // (pp, ww) is the next element in  $S^{i-1}$ 
                    while ((k <= h) and (pair[k].w < ww)) do
                        {
                            pair[next].p := pair[k].p;
                            pair[next].w := pair[k].w;
                            next := next + 1;
                            k := k + 1;
                        }
                    if ((k <= h) and (pair[k].w = ww)) then
                        {
                            if pp < pair[k].p then pp := pair[k].p;
                            k := k + 1;
                        }
                    if pp > pair[next-1].p then
                        {
                            pair[next].p := pp;

```

```

        pair[next].w:=ww;
        next:=next+ 1;
    }
    while ((k < h) and (pair[k].p< pair[next-1].p))
        do k:=k+ 1;
    }
    // Merge in remaining terms from S1~I
    while (k < h) do
    {
        pair[next].p:=pair[k].p; pair[next].w:=pair[k].w;
        next:=next+ 1;k :=k + 1;
    }
    // Initialize for Si+1.
    t :=h + 1;h :=next-1;b[i + 1]:=next;
}
TraceBack(p,w,pair,x,m,n);
}

```

Example 1: 0/1 knapsack problem with $n=3$, weights $(w_1, w_2, w_3)=(2, 3, 4)$ probability $(p_1, p_2, p_3)=(1, 2, 5)$ and $m=w=6$.

Solutions:

Method 1:

Initially $f_0(X)=0$, for all X and $f_i(X)=-\infty$ if $X < 0$.

$F_n(M) = \max \{f_{n-1}(M), f_{n-1}(M - w_n) + p_n\}$

$F_3(6) = \max \{f_2(6), f_2(6 - 4) + 5\} = \max \{f_2(6), f_2(2) + 5\}$

$F_2(6) = \max \{f_1(6), f_1(6 - 3) + 2\} = \max \{f_1(6), f_1(3) + 2\}$

$F_1(6) = \max \{f_0(6), f_0(6 - 2) + 1\} = \max \{0, 0 + 1\} = 1$

$F_1(3) = \max \{f_0(3), f_0(3 - 2) + 1\} = \max \{0, 0 + 1\} = 1$

Therefore, $F_2(6) = \max \{1, 1 + 2\} = 3$

$F_2(2) = \max \{f_1(2), f_1(2 - 3) + 2\} = \max \{f_1(2), -\infty + 2\}$

$F_1(2) = \max \{f_0(2), f_0(2 - 2) + 1\} = \max \{0, 0 + 1\} = 1$

$F_2(2) = \max \{1, -\infty + 2\} = 1$

Finally, $f_3(6) = \max \{3, 1 + 5\} = 6$

Method 2:

For the given data we have:

$S_0 = \{(0, 0)\}; S_0 = \{(1, 2)\}$

$S_1 = (S_0 \cup S_{01}) = \{(0, 0), (1, 2)\}$

$X - 2 = 0 \Rightarrow x = 2. y - 3 = 0 \Rightarrow y = 3$

$X - 2 = 1 \Rightarrow x = 3. y - 3 = 2 \Rightarrow y = 5$

$S_{11} = \{(2, 3), (3, 5)\}$

$S_2 = (S_1 \cup S_{11}) = \{(0, 0), (1, 2), (2, 3), (3, 5)\}$

$X - 5 = 0 \Rightarrow x = 5. y - 4 = 0 \Rightarrow y = 4$

$X - 5 = 1 \Rightarrow x = 6. y - 4 = 2 \Rightarrow y = 6$

$X - 5 = 2 \Rightarrow x = 7. y - 4 = 3 \Rightarrow y = 7$

$X - 5 = 3 \Rightarrow x = 8. y - 4 = 5 \Rightarrow y = 9$

$S_{21} = \{(5, 4), (6, 6), (7, 7), (8, 9)\}$

$S_3 = (S_2 \cup S_{21}) = \{(0, 0), (1, 2), (2, 3), (3, 5), (5, 4), (6, 6), (7, 7), (8, 9)\}$

9)}}

By applying Dominance rule,

$S_3 = (S_2 \cup S_{21}) = \{(0, 0), (1, 2), (2, 3), (5, 4), (6, 6)\}$

From (6, 6) we can infer that the maximum Profit $\pi_{xi} = 6$ and weight $\pi_{wi} = 6$

Method 3:

Method 3(tabular method)

Weights $(w_1, w_2, w_3) = (2, 3, 4)$ Profits $(p_1, p_2, p_3) = (1, 2, 5)$ M or $W = 6$

```

For(int i=0 ; i<=n; i++)
{
    For(int w=0; w<=M || W; w++)
    {
        If(i==0 || w==0) // 1st condition
            K[i][w]=0
        Else if(w[i]<=W) // 2nd condition
            K[i][w]=MAX((p[i]+k[i-1][w-w[i]]), k[i-1][w])
        Else // 3rd condition
            K[i][w]=k[i-1][w]
    }
}

```

Solution

First create a table with weights starting from 0 to M or W i.e 0 to 6 and with number of weights ie n 3 starting from 0 to 3

profit	weight		WEIGHTS (W)						
		w[i]	0	1	2	3	4	5	6
0	0	0							
1	2	1							
2	3	2							
5	4	3							

Based on the condition 1 the values are generated as $i=0$ or $w=0$

profit	weight		WEIGHTS (W)							
		w[i]	0	1	2	3	4	5	6	
0	0	0	0	0	0	0	0	0	0	
1	2	1	0							
2	3	2	0							
5	4	3	0							

Based on the condition 3 the values are generated as

$$K[i][w]=k[i-1][w] \Rightarrow k[1][1]=k[1-1][1]=K[0][1]=k[1][1]=0$$

			WEIGHTS (W)						
profit	weight	w[i]	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0
1	2	1	0	0					
2	3	2	0						
5	4	3	0						

Now $i=1$ and $W=2$

$$K[i][w]=\text{MAX}((p[i]+k[i-1][w-w[i]]),k[i-1][w])$$

$$K[1][2]=\text{MAX}((p[1]+k[1-1][2-w[1]]),k[1-1][2]) \Rightarrow \text{MAX}(1+k[0][2-2],k[0][2])$$

$$\Rightarrow \text{MAX}(1+0,0)$$

$$\Rightarrow \text{MAX}(1,0)$$

$$\Rightarrow 1$$

$$\text{Now } i=1, W=3 \Rightarrow K[1][3]=\text{MAX}((p[1]+k[1-1][3-w[1]]),k[1-1][3]) \Rightarrow \text{MAX}(1+0,0) \Rightarrow 1$$

$$\text{Now } i=1, W=4 \Rightarrow K[1][4]=\text{MAX}((p[1]+k[1-1][4-w[1]]),k[1-1][4]) \Rightarrow \text{MAX}(1+0,0) \Rightarrow 1$$

$$\text{Now } i=1, W=5 \Rightarrow K[1][5]=\text{MAX}((p[1]+k[1-1][5-w[1]]),k[1-1][5]) \Rightarrow \text{MAX}(1+0,0) \Rightarrow 1$$

$$\text{Now } i=1, W=6 \Rightarrow K[1][6]=\text{MAX}((p[1]+k[1-1][6-w[1]]),k[1-1][6]) \Rightarrow \text{MAX}(1+0,0) \Rightarrow 1$$

WEIGHTS (W)						
-------------	--	--	--	--	--	--

profit	weight	w[i]	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1
2	3	2	0						
5	4	3	0						

Based on the condition 3

$i=2, w=1$

$K[i][w]=K[i-1][w] \Rightarrow K[2][1]=K[2-1][1] \Rightarrow K[1][1] \Rightarrow 0$

$i=2, w=2 \Rightarrow K[2][2]=K[2-1][2] \Rightarrow K[1][2] \Rightarrow 1$

Based on the condition 2

$K[i][w]=\text{MAX}((p[i]+K[i-1][w-w[i]]), K[i-1][w])$

$i=2, w=3 \quad K[2][3]=\text{MAX}(2+0, 1) \Rightarrow 2$

$i=2, w=4 \quad K[2][4]=\text{MAX}(2+1, 1) \Rightarrow 2$

$i=2, w=5 \quad K[2][5]=\text{MAX}(2+1, 1) \Rightarrow 3$

$i=2, w=6 \quad K[2][6]=\text{MAX}(2+1, 1) \Rightarrow 3$

profit	weight	WEIGHTS (W)							
		w[i]	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3
5	4	3	0						

Based on the condition 3

$K[i][w]=K[i-1][w]$

Now $i=3$ and $w=1 \Rightarrow K[3][1]=K[3-1][1] \Rightarrow 0$

Now $i=3$ and $w=2 \Rightarrow K[3][2]=K[3-1][2] \Rightarrow 1$

Now $i=3$ and $w=3 \Rightarrow K[3][3]=K[3-1][3] \Rightarrow 2$

Based on the condition 2

$K[i][w]=\text{MAX}((p[i]+K[i-1][w-w[i]]), K[i-1][w])$

Now $i=3, w=4 \Rightarrow K[3][4] = \text{MAX}((p[3]+k[3-1][4-w[3]]), k[3-1][4]) \Rightarrow \text{MAX}(5+k[2,0], 3) \Rightarrow 5$

Now $i=3, w=5 \Rightarrow K[3][5] = \text{MAX}((p[3]+k[3-1][5-w[3]]), k[3-1][5]) \Rightarrow \text{MAX}(5+k[2,1], 3) \Rightarrow 5$

Now $i=3, w=6 \Rightarrow K[3][6] = \text{MAX}((p[3]+k[3-1][6-w[3]]), k[3-1][6]) \Rightarrow \text{MAX}(5+k[2,2], 3) \Rightarrow 6$

profit	weight		WEIGHTS (W)						
		w[i]	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3
5	4	3	0	0	1	2	5	5	6

Now we can see that $k(3,6)=6$ which is filled, which is generated by considering the values of $k(2,2)=1$ and this values also exists in the $[1,2]$.

So the values to be considered are $k[3,6] \Rightarrow w[3]=4$ And $k[1,2] \Rightarrow w[1]=2$

Then $[x_1, x_2, x_3] = [1, 0, 1]$

By using the max Weight $\Rightarrow w_1x_1 + w_2x_2 + w_3x_3 \Rightarrow 2*1 + 3*0 + 4*1 \Rightarrow 4$

By using the max Profit $\Rightarrow p_1x_1 + p_2x_2 + p_3x_3 \Rightarrow 1*1 + 3*0 + 5*1 \Rightarrow 6$

Example 2: $N=4, (W_1, W_2, W_3, W_4) = (10, 15, 6, 9), (P_1, P_2, P_3, P_4) = (2, 5, 8, 1)$ and $W=M=21$

Solution:

Initially $S^0 = \{(0, 0)\}$

$$S^0_1 = \{((0+p_1), (0+w_1))\} \\ = \{((0+2), (0+10))\} = \{(2, 10)\}$$

Now calculate S^1 , which is obtained by merging S^0 with S^0_1

$$\square S^1 = \{(0, 0), (2, 10)\}$$

Now calculate the $S^1_1 = \{(0, 0), (2, 10)\} + (p_2, w_2)$

$$= \{(0+5, 0+15), (2+5, 10+15)\} \\ = \{(5, 15), (7, 25)\}$$

Now calculate S^2 , which is obtained by merging S^1 with S^1_1

$$\square S^2 = \{(0, 0), (2, 10), (5, 15), (7, 25)\}$$

Now calculate the $S^2_1 = \{(0, 0), (2, 10), (5, 15), (7, 25)\} + (p_3, w_3)$

$$= \{(0+8, 0+6), (2+8, 10+6), (5+8, 15+6), (7+8, 25+6)\} \\ = \{(8, 6), (10, 16), (13, 21), (15, 31)\}$$

No calculate S^3 , which is obtained by merging S^2 with S^2_1

$$\square S^3 = \{(0, 0), (2, 10), (5, 15), (7, 25), (8, 6), (10, 16), (13, 21), (15, 31)\}$$

Now calculate the

$$\begin{aligned} S^3_1 &= \{(0, 0), (2, 10), (5, 15), (7, 25), (8, 6), (10, 16), (13, 21), (15, 31)\} + (p4, w4) \\ &= \{(0+1, 0+9), (2+1, 10+9), (5+1, 15+9), (7+1, 25+9), (8+1, 6+9), (10+1, 16+9), (13+1, 21+9), (15+1, 31+9)\} \\ &= \{(1, 9), (3, 9), (6, 24), (8, 34), (9, 15), (11, 25), (14, 30), (16, 40)\} \end{aligned}$$

No calculate S^4 , which is obtained by merging S^3 with S^3_1

$$\square S^4 = \{(0, 0), (2, 10), (5, 15), (7, 25), (8, 6), (10, 16), (13, 21), (15, 31), (1, 9), (3, 9), (6, 24), (8, 34), (9, 15), (11, 25), (14, 30), (16, 40)\}$$

The pair (8,34) is perged from the knapsack

Finally the set is $S^4 = \{(0, 0), (2, 10), (5, 15), (7, 25), (8, 6), (10, 16), (13, 21), (15, 31), (1, 9), (3, 9), (6, 24), (9, 15), (11, 25), (14, 30), (16, 40)\}$

Here $m=21$

$\Rightarrow (13, 21) \in S^4$ and $\in S^3$ therefore $X_4=1$

$\Rightarrow (13, 21) \in S^3$ and $\in S^2$ therefore $X_3=1$

Now we the pair (13,21) by adding the values (5,15) to (8,6)

$(5, 15) \in S^3$ and $\in S^2$ therefore $X_2=1$

Substitute the values (5,15) from (5,15) $\Rightarrow (0,0)$ therefore $X_1=0$

Hence the optimal solution is $(x_1, x_2, x_3, x_4) = (0, 1, 1, 1) = \sum_{i=1}^3 P_i X_i = 14$

Complexity Analysis:

- Time Complexity: $O(2^n)$. As there are redundant sub problems.
- Auxiliary Space: $O(1)$. As no extra data structure has been used for storing values

ALL PAIRS SHORTEST PATH

In the all pairs shortest path problem, we are to find a shortest path between every pair of vertices in a directed graph G . That is, for every pair of vertices (i, j) , we are to find a shortest path from i to j as well as one from j to i . These two paths are the same when G is undirected.

When no edge has a negative length, the all-pairs shortest path problem may be solved by using Dijkstra's greedy single source algorithm n times, once with each of the n vertices as the source vertex.

The all pairs shortest path problem is to determine a matrix A such that $A(i, j)$ is the length of a shortest path from i to j . The matrix A can be obtained by solving n single-source problems using the algorithm shortest Paths. Since each application of this procedure requires $O(n^2)$ time, the matrix A can be obtained in $O(n^3)$ time.

The dynamic programming solution, called Floyd's algorithm, runs in $O(n^3)$ time. Floyd's algorithm works even when the graph has negative length edges (provided there are no negative length cycles).

The shortest i to j path in G , $i \neq j$ originates at vertex i and goes through some intermediate vertices (possibly none) and terminates at vertex j . If k is an intermediate vertex on this shortest path, then the subpaths from i to k and from k to j must be shortest paths from i to k and k to j , respectively. Otherwise, the i to j path is not of minimum length. So, the principle of optimality holds. Let $A_k(i, j)$ represent the length of a shortest path from i to j going through no vertex of index greater than k , we obtain:

$$A_k(i, j) = \{\min \{\min_{1 \leq k \leq n} \{A_{k-1}(i, k) + A_{k-1}(k, j)\}, c(i, j)\}$$

Algorithm All Paths (Cost, A, n)

// cost [1:n, 1:n] is the cost adjacency matrix of a graph which

// n vertices; A [i, j] is the cost of a shortest path from vertex

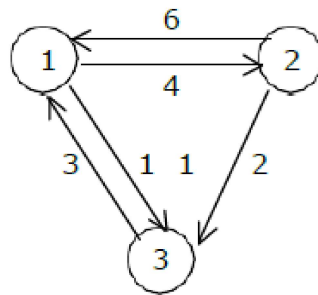
// i to vertex j.

cost [i, i] = 0.0, for $1 < i < n$.

```
{
    for i := 1 to n do
        for j:= 1 to n do
            A [i, j] := cost [i, j]; // copy cost into A.
        for k := 1 to n do
            for i := 1 to n do
                for j := 1 to n do
                    A [i, j] := min (A [i, j], A [i, k] + A [k, j]);
            }
}
```

Complexity Analysis: A Dynamic programming algorithm based on this recurrence involves in calculating $n+1$ matrices, each of size $n \times n$. Therefore, the algorithm has a complexity of $O(n^3)$.

Example 1: Given a weighted digraph $G = (V, E)$ with weight. Determine the length of the shortest path between all pairs of vertices in G . Here we assume that there are no cycles with zero or negative cost



Cost of $A^0 = [0 \ 4 \ 11 \ 6 \ 0 \ 2 \ 3 \ \infty \ 0]$

General formula: $\min_{1 \leq k \leq n} \{A^{k-1}(i, k) + A^{k-1}(k, j)\}, c(i, j)\}$

Solve the problem for different values of $k = 1, 2$ and 3

Step 1: Solving the equation for, $k = 1$;

$$\begin{aligned} A^1(1, 1) &= \min \{A^0(1, 1) + A^0(1, 1)\}, c(1, 1)\} = \min \{0 + 0, 0\} = 0 \\ A^1(1, 2) &= \min \{A^0(1, 1) + A^0(1, 2)\}, c(1, 2)\} = \min \{(0 + 4), 4\} = 4 \\ A^1(1, 3) &= \min \{A^0(1, 1) + A^0(1, 3)\}, c(1, 3)\} = \min \{(0 + 11), 11\} = 11 \\ A^1(2, 1) &= \min \{A^0(2, 1) + A^0(1, 1)\}, c(2, 1)\} = \min \{(6 + 0), 6\} = 6 \\ A^1(2, 2) &= \min \{A^0(2, 1) + A^0(1, 2)\}, c(2, 2)\} = \min \{(6 + 4), 0\} = 0 \\ A^1(2, 3) &= \min \{A^0(2, 1) + A^0(1, 3)\}, c(2, 3)\} = \min \{(6 + 11), 2\} = 2 \\ A^1(3, 1) &= \min \{A^0(3, 1) + A^0(1, 1)\}, c(3, 1)\} = \min \{(3 + 0), 3\} = 3 \\ A^1(3, 2) &= \min \{A^0(3, 1) + A^0(1, 2)\}, c(3, 2)\} = \min \{(3 + 4), \infty\} = 7 \\ A^1(3, 3) &= \min \{A^0(3, 1) + A^0(1, 3)\}, c(3, 3)\} = \min \{(3 + 11), 0\} = 0 \end{aligned}$$

Cost of $A^1 = [0 \ 4 \ 11 \ 6 \ 0 \ 2 \ 3 \ 7 \ 0]$

Step 2: Solving the equation for, $K = 2$;

$$\begin{aligned} A^2(1, 1) &= \min \{A^1(1, 2) + A^1(2, 1), c(1, 1)\} = \min \{(4 + 6), 0\} = 0 \\ A^2(1, 2) &= \min \{A^1(1, 2) + A^1(2, 2), c(1, 2)\} = \min \{(4 + 0), 4\} = 4 \\ A^2(1, 3) &= \min \{A^1(1, 2) + A^1(2, 3), c(1, 3)\} = \min \{(4 + 2), 11\} = 6 \\ A^2(2, 1) &= \min \{A^1(2, 2) + A^1(2, 1), c(2, 1)\} = \min \{(0 + 6), 6\} = 6 \\ A^2(2, 2) &= \min \{A^1(2, 2) + A^1(2, 2), c(2, 2)\} = \min \{(0 + 0), 0\} = 0 \\ A^2(2, 3) &= \min \{A^1(2, 2) + A^1(2, 3), c(2, 3)\} = \min \{(0 + 2), 2\} = 2 \\ A^2(3, 1) &= \min \{A^1(3, 2) + A^1(2, 1), c(3, 1)\} = \min \{(7 + 6), 3\} = 3 \\ A^2(3, 2) &= \min \{A^1(3, 2) + A^1(2, 2), c(3, 2)\} = \min \{(7 + 0), 7\} = 7 \\ A^2(3, 3) &= \min \{A^1(3, 2) + A^1(2, 3), c(3, 3)\} = \min \{(7 + 2), 0\} = 0 \end{aligned}$$

Cost of $A^2 = [0 \ 4 \ 6 \ 6 \ 0 \ 2 \ 3 \ 7 \ 0]$

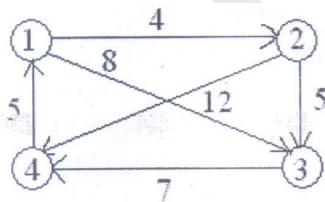
Step 3: Solving the equation for, $k = 3$;

$$\begin{aligned} A^3(1, 1) &= \min \{A^2(1, 3) + A^2(3, 1), c(1, 1)\} = \min \{(6 + 3), 0\} = 0 \\ A^3(1, 2) &= \min \{A^2(1, 3) + A^2(3, 2), c(1, 2)\} = \min \{(6 + 7), 4\} = 4 \\ A^3(1, 3) &= \min \{A^2(1, 3) + A^2(3, 3), c(1, 3)\} = \min \{(6 + 0), 6\} = 6 \\ A^3(2, 1) &= \min \{A^2(2, 3) + A^2(3, 1), c(2, 1)\} = \min \{(2 + 3), 6\} = 5 \\ A^3(2, 2) &= \min \{A^2(2, 3) + A^2(3, 2), c(2, 2)\} = \min \{(2 + 7), 0\} = 0 \\ A^3(2, 3) &= \min \{A^2(2, 3) + A^2(3, 3), c(2, 3)\} = \min \{(2 + 0), 2\} = 2 \\ A^3(3, 1) &= \min \{A^2(3, 3) + A^2(3, 1), c(3, 1)\} = \min \{(0 + 3), 3\} = 3 \\ A^3(3, 2) &= \min \{A^2(3, 3) + A^2(3, 2), c(3, 2)\} = \min \{(0 + 7), 7\} = 7 \\ A^3(3, 3) &= \min \{A^2(3, 3) + A^2(3, 3), c(3, 3)\} = \min \{(0 + 0), 0\} = 0 \end{aligned}$$

Cost of $A^3 = [0 \ 4 \ 6 \ 5 \ 0 \ 2 \ 3 \ 7 \ 0]$

The above matrix defines the shortest path between all the nodes in the graph.

Example 2: Given a weighted digraph $G = (V, E)$ with weight. Determine the length of the shortest path between all pairs of vertices in G . Here we assume that there are no cycles with zero or negative cost



Cost of $A^0 = [0 \ 4 \ 8 \ \infty \ 0 \ 5 \ 12 \ \infty \ 0 \ 7 \ 5 \ \infty \ 0]$

General formula: $\min_{1 \leq k \leq n} \{A^{k-1}(i, k) + A^{k-1}(k, j), c(i, j)\}$

Solve the problem for different values of $k = 1, 2, 3$ and 4

Step 1: Solving the equation for, $k = 1$;

$$\begin{aligned} A^1(1, 1) &= \min \{A^0(1, 1) + A^0(1, 1), c(1, 1)\} = \min \{0 + 0, 0\} = 0 \\ A^1(1, 2) &= \min \{A^0(1, 1) + A^0(1, 2), c(1, 2)\} = \min \{0 + 4, 4\} = 4 \\ A^1(1, 3) &= \min \{A^0(1, 1) + A^0(1, 3), c(1, 3)\} = \min \{0 + 8, 8\} = 8 \\ A^1(1, 4) &= \min \{A^0(1, 1) + A^0(1, 4), c(1, 4)\} = \min \{0 + \infty, \infty\} = \infty \end{aligned}$$

$$A^1(2, 1) = \min \{A^0(2, 1) + A^0(1, 1), c(2, 1)\} = \min \{\infty + 0, \infty\} = \infty$$

$$\begin{aligned}
 A^1(2,2) &= \min \{(A^0(2,1) + A^0(2,2)), c(2,2)\} = \min \{\infty + 0, 0\} = 0 \\
 A^1(2,3) &= \min \{(A^0(2,1) + A^0(1,3)), c(2,3)\} = \min \{\infty + 8, 5\} = 5 \\
 A^1(2,4) &= \min \{(A^0(2,1) + A^0(1,4)), c(2,4)\} = \min \{\infty + \infty, 12\} = 12
 \end{aligned}$$

$$\begin{aligned}
 A^1(3,1) &= \min \{(A^0(3,1) + A^0(1,1)), c(3,1)\} = \min \{\infty + 0, \infty\} = \infty \\
 A^1(3,2) &= \min \{(A^0(3,1) + A^0(1,2)), c(3,2)\} = \min \{\infty + 4, \infty\} = \infty \\
 A^1(3,3) &= \min \{(A^0(3,1) + A^0(1,3)), c(3,3)\} = \min \{\infty + 8, 0\} = 0 \\
 A^1(3,4) &= \min \{(A^0(3,1) + A^0(1,4)), c(3,4)\} = \min \{\infty + \infty, 7\} = 7
 \end{aligned}$$

$$\begin{aligned}
 A^1(4,1) &= \min \{(A^0(4,1) + A^0(1,1)), c(4,1)\} = \min \{5+0, 5\} = 5 \\
 A^1(4,2) &= \min \{(A^0(4,1) + A^0(1,2)), c(4,2)\} = \min \{5+4, \infty\} = 9 \\
 A^1(4,3) &= \min \{(A^0(4,1) + A^0(1,3)), c(4,3)\} = \min \{5+8, \infty\} = 13 \\
 A^1(4,4) &= \min \{(A^0(4,1) + A^0(1,4)), c(4,4)\} = \min \{0 + \infty, 0\} = 0
 \end{aligned}$$

Cost of $A^1 = [0 \ 4 \ 8 \ \infty \ \infty \ 0 \ 5 \ 12 \ \infty \ \infty \ 0 \ 7 \ 5 \ 9 \ 13 \ 0]$

Step 2: Solving the equation for, $k = 2$;

$$\begin{aligned}
 A^2(1,1) &= \min \{(A^1(1,2) + A^1(2,1)), c(1,1)\} = \min \{4 + \infty, 0\} = 0 \\
 A^2(1,2) &= \min \{(A^1(1,2) + A^1(2,2)), c(1,2)\} = \min \{4 + 0, 4\} = 4 \\
 A^2(1,3) &= \min \{(A^1(1,2) + A^1(2,3)), c(1,3)\} = \min \{4 + 5, 8\} = 8 \\
 A^2(1,4) &= \min \{(A^1(1,2) + A^1(2,4)), c(1,4)\} = \min \{4 + 12, \infty\} = 16
 \end{aligned}$$

$$\begin{aligned}
 A^2(2,1) &= \min \{(A^1(2,2) + A^1(2,1)), c(2,1)\} = \min \{0 + \infty, \infty\} = \infty \\
 A^2(2,2) &= \min \{(A^1(2,2) + A^1(2,2)), c(2,2)\} = \min \{0 + 0, 0\} = 0 \\
 A^2(2,3) &= \min \{(A^1(2,2) + A^1(2,3)), c(2,3)\} = \min \{0 + 5, 5\} = 5 \\
 A^2(2,4) &= \min \{(A^1(2,2) + A^1(2,4)), c(2,4)\} = \min \{0 + 12, 12\} = 12
 \end{aligned}$$

$$\begin{aligned}
 A^2(3,1) &= \min \{(A^1(3,2) + A^1(2,1)), c(3,1)\} = \min \{\infty + \infty, \infty\} = \infty \\
 A^2(3,2) &= \min \{(A^1(3,2) + A^1(2,2)), c(3,2)\} = \min \{\infty + 0, \infty\} = \infty \\
 A^2(3,3) &= \min \{(A^1(3,2) + A^1(2,3)), c(3,3)\} = \min \{\infty + 5, 0\} = 0 \\
 A^2(3,4) &= \min \{(A^1(3,2) + A^1(2,4)), c(3,4)\} = \min \{\infty + 12, 7\} = 7
 \end{aligned}$$

$$\begin{aligned}
 A^2(4,1) &= \min \{(A^1(4,2) + A^1(2,1)), c(4,1)\} = \min \{9 + \infty, 5\} = 5 \\
 A^2(4,2) &= \min \{(A^1(4,2) + A^1(2,2)), c(4,2)\} = \min \{9 + 0, 9\} = 9 \\
 A^2(4,3) &= \min \{(A^1(4,2) + A^1(2,3)), c(4,3)\} = \min \{9 + 5, 13\} = 13 \\
 A^2(4,4) &= \min \{(A^1(4,2) + A^1(2,4)), c(4,4)\} = \min \{9 + 12, 0\} = 0
 \end{aligned}$$

Cost of $A^2 = [0 \ 4 \ 8 \ 16 \ \infty \ 0 \ 5 \ 12 \ \infty \ \infty \ 0 \ 7 \ 5 \ 9 \ 13 \ 0]$

Step 3: Solving the equation for, $k = 3$;

$$\begin{aligned}
 A^3(1,1) &= \min \{(A^2(1,3) + A^2(3,1)), c(1,1)\} = \min \{8 + \infty, 0\} = 0 \\
 A^3(1,2) &= \min \{(A^2(1,3) + A^2(3,2)), c(1,2)\} = \min \{8 + \infty, 4\} = 4 \\
 A^3(1,3) &= \min \{(A^2(1,3) + A^2(3,3)), c(1,3)\} = \min \{8 + 0, 8\} = 8 \\
 A^3(1,4) &= \min \{(A^2(1,3) + A^2(3,4)), c(1,4)\} = \min \{8 + 7, 16\} = 15
 \end{aligned}$$

$$\begin{aligned}
 A^3(2,1) &= \min \{(A^2(2,3) + A^2(3,1)), c(2,1)\} = \min \{5 + \infty, \infty\} = \infty \\
 A^3(2,2) &= \min \{(A^2(2,3) + A^2(3,2)), c(2,2)\} = \min \{5 + \infty, 0\} = 0 \\
 A^3(2,3) &= \min \{(A^2(2,3) + A^2(3,3)), c(2,3)\} = \min \{5 + 0, 5\} = 5 \\
 A^3(2,4) &= \min \{(A^2(2,3) + A^2(3,4)), c(2,4)\} = \min \{5 + 7, 12\} = 12
 \end{aligned}$$

$$\begin{aligned}
A^3(3,1) &= \min \{(A^2(3,3) + A^2(3,1)), c(3,1)\} = \min \{0 + \infty, \infty\} = \infty \\
A^3(3,2) &= \min \{(A^2(3,3) + A^2(3,2)), c(3,2)\} = \min \{0 + \infty, \infty\} = \infty \\
A^3(3,3) &= \min \{(A^2(3,3) + A^2(3,3)), c(3,3)\} = \min \{0 + 0, 0\} = 0 \\
A^3(3,4) &= \min \{(A^2(3,3) + A^2(3,4)), c(3,4)\} = \min \{0 + 7, 7\} = 7
\end{aligned}$$

$$\begin{aligned}
A^3(4,1) &= \min \{(A^2(4,3) + A^2(3,1)), c(4,1)\} = \min \{13 + \infty, 5\} = 5 \\
A^3(4,2) &= \min \{(A^2(4,3) + A^2(3,2)), c(4,2)\} = \min \{13 + \infty, 9\} = 9 \\
A^3(4,3) &= \min \{(A^2(4,3) + A^2(3,3)), c(4,3)\} = \min \{13 + 0, 13\} = 13 \\
A^3(4,4) &= \min \{(A^2(4,3) + A^2(3,4)), c(4,4)\} = \min \{13 + 7, 0\} = 0
\end{aligned}$$

Cost of $A^3 = [0 \ 4 \ 8 \ 15 \ \infty \ 0 \ 5 \ 12 \ \infty \ \infty \ 0 \ 7 \ 5 \ 9 \ 13 \ 0]$

Step 4: Solving the equation for, $k = 4$;

$$\begin{aligned}
A^4(1,1) &= \min \{(A^3(1,4) + A^3(4,1)), c(1,1)\} = \min \{15 + 5, 0\} = 0 \\
A^4(1,2) &= \min \{(A^3(1,4) + A^3(4,2)), c(1,2)\} = \min \{15 + 9, 4\} = 4 \\
A^4(1,3) &= \min \{(A^3(1,4) + A^3(4,3)), c(1,3)\} = \min \{15 + 13, 8\} = 8 \\
A^4(1,4) &= \min \{(A^3(1,4) + A^3(4,4)), c(1,4)\} = \min \{15 + 0, 15\} = 15
\end{aligned}$$

$$\begin{aligned}
A^4(2,1) &= \min \{(A^3(2,4) + A^3(4,1)), c(2,1)\} = \min \{12 + 5, \infty\} = 17 \\
A^4(2,2) &= \min \{(A^3(2,4) + A^3(4,2)), c(2,2)\} = \min \{12 + 9, 0\} = 0 \\
A^4(2,3) &= \min \{(A^3(2,4) + A^3(4,3)), c(2,3)\} = \min \{12 + 13, 5\} = 5 \\
A^4(2,4) &= \min \{(A^3(2,4) + A^3(4,4)), c(2,4)\} = \min \{12 + 0, 12\} = 12
\end{aligned}$$

$$\begin{aligned}
A^4(3,1) &= \min \{(A^3(3,4) + A^3(4,1)), c(3,1)\} = \min \{7 + 5, \infty\} = 12 \\
A^4(3,2) &= \min \{(A^3(3,4) + A^3(4,2)), c(3,2)\} = \min \{7 + 9, \infty\} = 16 \\
A^4(3,3) &= \min \{(A^3(3,4) + A^3(4,3)), c(3,3)\} = \min \{7 + 13, 0\} = 0 \\
A^4(3,4) &= \min \{(A^3(3,4) + A^3(4,4)), c(3,4)\} = \min \{7 + 0, 7\} = 7
\end{aligned}$$

$$\begin{aligned}
A^4(4,1) &= \min \{(A^3(4,4) + A^3(4,1)), c(4,1)\} = \min \{0 + 5, 5\} = 5 \\
A^4(4,2) &= \min \{(A^3(4,4) + A^3(4,2)), c(4,2)\} = \min \{0 + 9, 9\} = 9 \\
A^4(4,3) &= \min \{(A^3(4,4) + A^3(4,3)), c(4,3)\} = \min \{0 + 13, 13\} = 13 \\
A^4(4,4) &= \min \{(A^3(4,4) + A^3(4,4)), c(4,4)\} = \min \{0 + 0, 0\} = 0
\end{aligned}$$

Cost of $A^4 = [0 \ 4 \ 8 \ 15 \ 17 \ 0 \ 5 \ 12 \ 12 \ 16 \ 0 \ 7 \ 5 \ 9 \ 13 \ 0]$

TRAVELING SALESPERSON PROBLEM

Let $G = (V, E)$ be a directed graph with edge costs C_{ij} . The variable c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \infty$ if $i, j \notin E$. Let $|V| = n$ and assume $n > 1$. A tour of G is a directed simple cycle that includes every vertex in V . The cost of a tour is the sum of the cost of the edges on the tour. The traveling salesperson's problem is to find a tour of minimum cost. The tour is to be a simple path that starts and ends at vertex 1.

Let $g(i, S)$ be the length of shortest path starting at vertex i , going through all vertices in S , and terminating at vertex 1. The function $g(1, V - \{1\})$ is the length of an optimal salesperson tour. From the principle of optimality it follows that:

$$g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{C_{1k} + g(k, V - \{1, k\})\} \quad \text{---1}$$

Generalizing equation 1, we obtain (for $i \notin S$)

$$g(i, S) = \min_{j \in S} \{C_{ij} + g(i, S - \{j\})\} \quad \text{---2}$$

The Equation can be solved for $g(1, V - \{1\})$ if we know $g(k, V - \{1, k\})$ for all choices of k .

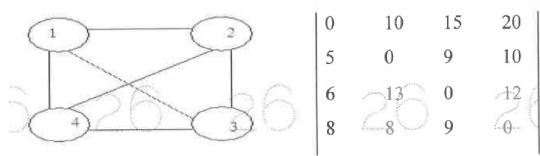
Equation 1 can be solved if we know the values of $g(k, V - \{1, k\})$ for all choices of k .

The value of g can be obtained from equation 2

$$g(i, \emptyset) = C_{i1}, \quad 1 < i \leq n \quad \text{---3}$$

** $g(i, S) \forall S$ of size 1, then $g(i, S)$ for s with $|s|=2$ and $|s| < n-1$ the value of i & s for which $g(i, S)$ is needed such that $i \neq 1, 1 \notin s$ and $i \notin s$

Example 1: Describe a travelling salesperson problem. Find the minimum cost tour for the following graph using the dynamic programming. Cost of the edges are given by matrix shown in below fig



Solution:

From the equation 3, we get:

$$g(2, \emptyset) = C_{21} = 5$$

$$g(3, \emptyset) = C_{31} = 6$$

$$g(4, \emptyset) = C_{41} = 8$$

From the equation 4, we get

$$g(2, \{3\}) = \min\{C_{23} + g(3, \emptyset)\} = 9 + 6 = 15$$

$$g(2, \{4\}) = C_{24} + g(4, \emptyset) = 10 + 8 = 18$$

$$g(3, \{2\}) = C_{32} + g(2, \emptyset) = 13 + 5 = 18$$

$$g(3, \{4\}) = C_{34} + g(4, \emptyset) = 12 + 8 = 20$$

$$g(4, \{2\}) = C_{42} + g(2, \emptyset) = 8 + 5 = 13$$

$$g(4, \{3\}) = C_{43} + g(3, \emptyset) = 9 + 6 = 15$$

Now we compare $g(i, s)$ with $|s| = 2, i \neq 1, 1 \notin s$ and $i \notin s$

$$g(2, \{3, 4\}) = \min\{C_{23} + g(3, \{4\}), C_{24} + g(4, \{3\})\} = \min\{9 + 20, 10 + 15\} = 25$$

$$g(3, \{2, 4\}) = \min\{C_{32} + g(2, \{4\}), C_{34} + g(4, \{2\})\} = \min\{13 + 18, 12 + 13\} = 25$$

$$g(4, \{2, 3\}) = \min\{C_{42} + g(2, \{3\}), C_{43} + g(3, \{2\})\} = \min\{8 + 15, 9 + 18\} = 23$$

From equation 1

$$g(1, \{2, 3, 4\}) = \min\{C_{12} + g(2, \{3, 4\}), C_{13} + g(3, \{2, 4\}), C_{14} + g(4, \{2, 3\})\} = \min\{10 + 25, 15 + 25, 20 + 23\} = 35$$

or

Let us start the tour from vertex 1:

$$g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V - \{1, k\})\} - (1)$$

More generally writing:

$$g(i, s) = \min \{c_{ij} + g(j, s - \{j\})\} - (2)$$

Clearly, $g(i, \emptyset) = c_{i1}$, $1 \leq i \leq n$. So,

$$g(2, \emptyset) = C_{21} = 5$$

$$g(3, \emptyset) = C_{31} = 6$$

$$g(4, \emptyset) = C_{41} = 8$$

Using equation - (2) we obtain:

$$g(1, \{2, 3, 4\}) = \min \{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\}$$

$$g(2, \{3, 4\}) = \min \{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\} = \min \{9 + g(3, \{4\}), 10 + g(4, \{3\})\}$$

$$g(3, \{4\}) = \min \{c_{34} + g(4, \emptyset)\} = 12 + 8 = 20$$

$$g(4, \{3\}) = \min \{c_{43} + g(3, \emptyset)\} = 9 + 6 = 15$$

$$\text{Therefore, } g(2, \{3, 4\}) = \min \{9 + 20, 10 + 15\} = \min \{29, 25\} = 25$$

$$g(3, \{2, 4\}) = \min \{(c_{32} + g(2, \{4\})), (c_{34} + g(4, \{2\}))\}$$

$$g(2, \{4\}) = \min \{c_{24} + g(4, \emptyset)\} = 10 + 8 = 18$$

$$g(4, \{2\}) = \min \{c_{42} + g(2, \emptyset)\} = 8 + 5 = 13$$

$$\text{Therefore, } g(3, \{2, 4\}) = \min \{13 + 18, 12 + 13\} = \min \{41, 25\} = 25$$

$$g(4, \{2, 3\}) = \min \{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\}$$

$$g(2, \{3\}) = \min \{c_{23} + g(3, \emptyset)\} = 9 + 6 = 15$$

$$g(3, \{2\}) = \min \{c_{32} + g(2, \emptyset)\} = 13 + 5 = 18$$

$$\text{Therefore, } g(4, \{2, 3\}) = \min \{8 + 15, 9 + 18\} = \min \{23, 27\} = 23$$

$$\begin{aligned} g(1, \{2, 3, 4\}) &= \min \{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\} \\ &= \min \{10 + 25, 15 + 25, 20 + 23\} \\ &= \min \{35, 40, 43\} = 35 \end{aligned}$$

An optimal tour of the graph of has length 35. A tour of this length can be constructed if we retain with each $g(i, S)$.

$g(1, \{2, 3, 4\}) = 1 \rightarrow 2$ (minimum i.e., $10 + 25$). Thus the tour starts from node 1 and goes to node 2.

$g(2, \{3, 4\}) = 2 \rightarrow 3$ (minimum i.e., $10 + 15$). Thus the tour starts from node 2 and goes to node 4.

$g(4, \{3\}) = 4 \rightarrow 3$ (minimum i.e., $9 + 6$). Thus the tour starts from node 4 and goes to node 3.

$g(3, \emptyset) = 3 \rightarrow 1$.

Therefore the optimal tour is 1, 2, 4, 3, 1

An algorithm that proceeds to find an optimal tour will require $O(n^2 2^n)$.

RELIABILITY DESIGN

The problem is to design a system that is composed of several devices connected in series. Let r_i be the reliability of device D_i (that is r_i is the probability that device i will function properly) then the reliability of the entire system is $\prod r_i$. Even if the individual devices are very reliable (the r_i 's are very close to one), the reliability of the system may not be very good.

For example,
if $n = 10$ and $r_i = 0.99$, $i \leq i \leq 10$,
then $\prod r_i = .904$.

Hence, it is desirable to duplicate devices. Multiple copies of the same .Device type are connected in parallel.

If stage i contains m_i copies of device D_i . Then the probability that all m_i have a malfunction is $(1 - r_i)^{m_i}$. Hence the reliability of stage i becomes $1 - (1 - r_i)^{m_i}$.

Stage 1 stage 2 stage 3 stage n

The reliability of stage 'i' is given by a function $\phi_i(m_i)$.

Therefore the reliability of the stages becomes $\prod_{i=1}^n \phi_i(m_i)$

Our problem is to use device duplication. This maximization is to be carried out under a cost constraint. Let c_i be the cost of each unit of device i and let c be the maximum allowable cost of the system being designed.

Let C_i be the cost of each unit of devices d_i & c be the maximum allowable cost of the entire system.

Maximize $\prod_{i=1}^n \phi_i(m_i)$

□ Subject to $\sum_{i=1}^n C_i m_i \leq c$ ----- (1)

□ $m_i \geq 1$ and integer $1 \leq i \leq n$

By dynamic programming ,we can assume each $c_i > 0$ and each m_i must be in the range

$1 \leq m_i \leq u_i$

where $u_i = \left\lfloor \frac{(C + C_i - \sum_{j=1}^N C_j)}{C_i} \right\rfloor$

u_i is the upper bound from the observation that is $m_j \geq 1$

The principal of optimality hold and

$F_n(C) = \max_{1 \leq m_n \leq u_n} \{ \phi_n(m_n) f_{n-1}(c - c_n m_n) \}$ ----- (2)

For any $F_i(X)$ & $i \geq 1$ equation 2 can be generalized to

$F_n(C) = \max_{1 \leq m_i \leq u_i} \{ \phi_i(m_i) f_{i-1}(c - c_i m_i) \}$ ----- (3)

Clearly $F_0(x) = 1$ for all x .

Let S_i consist of tuples of the form (f, x) , where $f = f_i(x)$.

There is at most one tuple for each different „x“, that result from a sequence of decisions on $m_1, m_2, \dots, m_n$. The dominance rule (f_1, x_1) dominate (f_2, x_2) if $f_1 \geq f_2$ and $x_1 \leq x_2$. Hence, dominated tuples can be discarded from S_i .

Example 1:

Design a three stage system with device types D_1, D_2 and D_3 . The costs are \$30, \$15 and \$20 respectively. The Cost of the system is to be no more than \$105. The reliability of each device is 0.9, 0.8 and 0.5 respectively.

Solution:

We assume that if stage I has m_i devices of type i in parallel, then $\varphi_i(m_i) = 1 - (1 - r_i)^{m_i}$

Since, we can assume each $c_i > 0$, each m_i must be in the range $1 \leq m_i \leq u_i$. Where:

$$u_i = \left\lceil \frac{(C + C_i - \sum_{j=1}^N c_j)}{c_i} \right\rceil$$

Using the above equation compute u_1 , u_2 and u_3 .

$$U_1 = (105 + 30 - (30 + 15 + 20)) / 30 = 70 / 30 = 2$$

$$U_2 = (105 + 15 - (30 + 15 + 20)) / 30 = 70 / 15 = 3$$

$$U_3 = (105 + 20 - (30 + 15 + 20)) / 30 = 70 / 20 = 3$$

We use $S_j^i = \{f_0(x), x\}$ initially $f_0(x) = 1$ and $x = 0$, $S^0 = \{1, 0\}$

Compute S^1 , S^2 and S^3 as follows :

$$S^1 = \text{depends on } U_1 = 2, \text{ so } S^1 = \{S_{1,1}^1, S_{1,2}^1\}$$

$$S^2 = \text{depends on } U_2 = 3, \text{ so } S^2 = \{S_{2,1}^2, S_{2,2}^2, S_{2,3}^2\}$$

$$\text{Now find } S^1 = \{(f_1(x), x)\}$$

$$f_1(x) = \{\varphi_1(1) f_0(\cdot), \varphi_1(2) f_0(\cdot)\} \text{ With devices } m_1 = 1 \text{ and } m_2 = 2 \text{ Compute } \varphi_1(1)$$

$$\text{and } \varphi_1(2) \text{ using the formula: } \varphi_i(m_i) = 1 - (1 - r_i)^{m_i}$$

$$\varphi_1(1) = 1 - (1 - r_1)^{m_1} = 1 - (1 - 0.9)^1 = 0.9$$

$$\varphi_1(2) = 1 - (1 - 0.9)^2 = 0.99$$

$$S^1 = \{f_1(x), x\} = \{(0.9, 30)\}$$

$$S^1 = \{0.99, 30 + 30\} = \{(0.99, 60)\}$$

$$\text{Therefore, } S^1 = \{(0.9, 30), (0.99, 60)\}$$

$$\text{Next find } S^2 = \{(f_2(x), x)\}$$

$$f_2(x) = \{\varphi_2(1) * f_1(\cdot), \varphi_2(2) * f_1(\cdot), \varphi_2(3) * f_1(\cdot)\}$$

$$\varphi_2(1) = 1 - (1 - r_1) = 1 - (1 - 0.8) = 1 - 0.2 = 0.8$$

$$\varphi_2(2) = 1 - (1 - 0.8)^2 = 0.96$$

$$\varphi_2(3) = 1 - (1 - 0.8)^3 = 0.992$$

$$\Sigma 12 = \{(0.8(0.9), 30 + 15), (0.8(0.99), 60 + 15)\} = \{(0.72, 45), (0.792, 75)\}$$

$$\begin{aligned} \Sigma 2 &= \{(0.96(0.9), 30 + 15 + 15), (0.96(0.99), 60 + 15 + 15)\} \\ &= \{(0.864, 60), (0.9504, 90)\} \end{aligned}$$

$$\begin{aligned} S^2 &= \{(0.992(0.9), 30 + 15 + 15 + 15), (0.992(0.99), 60 + 15 + 15 + 15)\} \\ &= \{(0.8928, 75), (0.98208, 105)\} \end{aligned}$$

$$S^2 = \{S_{1,1}^2, S_{2,2}^2, S_{3,3}^2\}$$

By applying Dominance rule to S^2 :

$$\text{Therefore, } S^2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$$

Dominance Rule:

If S^i contains two pairs (f_1, x_1) and (f_2, x_2) with the property that $f_1 \geq f_2$ and $x_1 \leq x_2$, then (f_1, x_1) dominates (f_2, x_2) , hence by dominance rule (f_2, x_2) can be discarded. Discarding or pruning rules such as the one above is known as dominance rule. Dominating tuples will be present in S^i and Dominated tuples has to be discarded from S^i .

Case 1: if $f_1 \leq f_2$ and $x_1 > x_2$ then discard (f_1, x_1) Case 2: if $f_1 \geq f_2$ and $x_1 < x_2$ the discard (f_2, x_2) Case 3: otherwise simply write (f_1, x_1)

$$S_2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$$

$$\varphi_3(1) = 1 - (1 - r_1)^{m_1} \\ = 1 - (1 - 0.5)^1 = 1 - 0.5 = 0.5$$

$$\varphi_3(2) = 1 - (1 - 0.5)^2 = 0.75$$

$$\varphi_3(3) = 1 - (1 - 0.5)^3 = 0.875$$

$$S^3 = \{(0.5(0.72), 45 + 20), (0.5(0.864), 60 + 20), (0.5(0.8928), 75 + 20)\}$$

$$S^3 = \{(0.36, 65), (0.437, 80), (0.4464, 95)\}$$

$$S_2^3 = \{(0.75(0.72), 45 + 20 + 20), (0.75(0.864), 60 + 20 + 20), (0.75(0.8928), 75 + 20 + 20)\}$$

$$= \{(0.54, 85), (0.648, 100), (0.6696, 115)\}$$

$$S_3^3 = \{(0.875(0.72), 45 + 20 + 20 + 20), (0.875(0.864), 60 + 20 + 20 + 20), (0.875(0.8928), 75 + 20 + 20 + 20)\}$$

$$S^3 = \{(0.63, 105), (1.756, 120), (0.7812, 135)\}$$

If cost exceeds 105, remove that tuples

$$S^3 = \{(0.36, 65), (0.437, 80), (0.54, 85), (0.648, 100)\}$$

The best design has a reliability of 0.648 and a cost of 100. Tracing back for the solution through S^i „s we can determine that $m_3 = 2$, $m_2 = 2$ and $m_1 = 1$.

Another solution

$$f_3(105) = \max \{\varphi_3(m_3) \cdot f_2(105 - 20m_3)\}$$

$$1 \leq m_3 \leq u_3$$

$$= \max \{\varphi_3(1) f_2(105 - 20), \varphi_3(2) f_2(105 - 20 \times 2), \varphi_3(3) f_2(105 - 20 \times 3)\}$$

$$= \max \{0.5 f_2(85), 0.75 f_2(65), 0.875 f_2(45)\}$$

$$= \max \{0.5 \times 0.8928, 0.75 \times 0.864, 0.875 \times 0.72\} =$$

$$0.648. f_2(85) = \max \{\varphi_2(m_2) \cdot f_1(85 - 15m_2)\}$$

$$1 \leq m_2 \leq u_2$$

$$= \max \{\varphi_2(1) \cdot f_1(85 - 15), \varphi_2(2) \cdot f_1(85 - 15 \times 2), \varphi_2(3) \cdot f_1(85 - 15 \times 3)\}$$

$$= \max \{0.8 f_1(70), 0.96 f_1(55), 0.992 f_1(40)\}$$

$$= \max \{0.8 \times 0.99, 0.96 \times 0.9, 0.99 \times 0.9\} =$$

$$\begin{aligned}
0.8928 \quad f_1(70) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(70 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(70 - 30), \phi_1(2) f_0(70 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times 1\} = \max \{0.9, 0.99\} = 0.99 \\
f_1(55) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(55 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(55 - 30), \phi_1(2) f_0(55 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times -\infty\} = \max \{0.9, -\infty\} = 0.9 \\
f_1(40) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(40 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(40 - 30), \phi_1(2) f_0(40 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times -\infty\} = \max \{0.9, -\infty\} = 0.9 \\
f_2(65) &= \max_{1 \leq m_2 \leq u_2} \{\phi_2(m_2) \cdot f_1(65 - 15m_2)\} \\
&= \max \{\phi_2(1) f_1(65 - 15), \phi_2(2) f_1(65 - 15 \times 2), \phi_2(3) f_1(65 - 15 \times 3)\} \\
&= \max \{0.8 f_1(50), 0.96 f_1(35), 0.992 f_1(20)\} \\
&= \max \{0.8 \times 0.9, 0.96 \times 0.9, -\infty\} = 0.864 \\
f_1(50) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(50 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(50 - 30), \phi_1(2) f_0(50 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times -\infty\} = \max \{0.9, -\infty\} = 0.9 \\
f_1(35) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(35 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(35 - 30), \phi_1(2) f_0(35 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times -\infty\} = \max \{0.9, -\infty\} = 0.9 \\
f_1(20) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(20 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(20 - 30), \phi_1(2) f_0(20 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times -\infty, \phi_1(2) \times -\infty\} = \max \{-\infty, -\infty\} \\
&= -\infty \\
f_2(45) &= \max_{1 \leq m_2 \leq u_2} \{\phi_2(m_2) \cdot f_1(45 - 15m_2)\} \\
&= \max \{\phi_2(1) f_1(45 - 15), \phi_2(2) f_1(45 - 15 \times 2), \phi_2(3) f_1(45 - 15 \times 3)\} \\
&= \max \{0.8 f_1(30), 0.96 f_1(15), 0.992 f_1(0)\} \\
&= \max \{0.8 \times 0.9, 0.96 \times -\infty, 0.99 \times -\infty\} = 0.72 \\
f_1(30) &= \max_{1 \leq m_1 \leq u_1} \{\phi_1(m_1) \cdot f_0(30 - 30m_1)\} \\
&= \max \{\phi_1(1) f_0(30 - 30), \phi_1(2) f_0(30 - 30 \times 2)\} \\
&= \max \{\phi_1(1) \times 1, \phi_1(2) \times -\infty\} = \max \{0.9, -\infty\} = 0.9
\end{aligned}$$

Similarly, $f_1(15) = -\infty$, $f_1(0) = -\infty$.

The best design has a reliability = 0.648 and

Cost = $30 \times 1 + 15 \times 2 + 20 \times 2 = 100$.

Tracing back for the solution through S^i 's we can determine that: $m_3 = 2$, $m_2 = 2$ and $m_1 = 1$.

PREVIOUS QUESTION

1. What do you mean by forward and backward approach of problem solving in Dynamic Programming?
2. Differentiate between greedy method and dynamic programming.
3. What you mean by dynamic programming

OBST

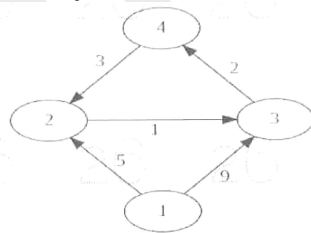
1. Define optimal binary search tree with an example.
2. Draw an Optimal Binary Search Tree for $n=4$ identifiers $(a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$
 $P(1:4) = (3, 3, 1, 1)$ and $Q(0:4) = (2, 3, 1, 1, 1)$.
3. Use function OBST(Algorithm 5.5) to compute $w(i, j)$, $r(i, j)$, and $c(i, j)$, $0 < i < j < 4$, for the identifier set $(01, 02, 03, 04)$ (cout, float, if, while) with $p(1) = 1/20$, $p(2) = 1/5$, $p(3) = 1/10$, $p(4) = 1/20$, $q(0) = 1/5$, $g(1) = 1/10$, $q(2) = 1/5$, $g(3) = 1/20$, and $g(4) = 1/20$. Using the $r(302 \setminus 253, j)$'s, construct the optimal binary search tree.

0/1 knapsack

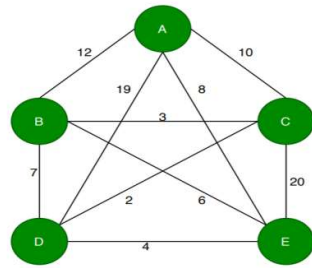
1. Define merging and purging rules in 0/1 knapsack problem.
2. Find an optimal solution to the 0/1 knapsack instance $n=7$, $m=15$, $(p_1, p_2, \dots, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and $(w_1, w_2, \dots, w_7) = (2, 3, 5, 7, 1, 4, 1)$
3. Find an optimal solution to the knapsack instance $n=4$, $m=15$, $(p_1, p_2, \dots, p_7) = (10, 5, 7, 11)$ and $(w_1, w_2, \dots, w_7) = (3, 4, 3, 5)$
4. What is the importance of knapsack algorithm in our daily life?
5. Explain the Knapsack problem with an example.
6. Solve the following 0/1 Knapsack problem using dynamic programming $P = (11, 21, 31, 33)$, $W = (2, 12, 23, 15)$, $C=42$, $n=4$.

ALL PAIRS SHORTEST PATH

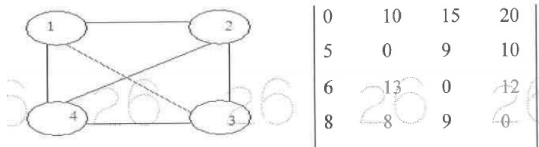
1. Write an algorithm of all pairs shortest path problem.
2. Discuss about all pairs shortest path problem with suitable example
3. With the help of suitable example explain the all pairs shortest path
4. Compute the shortest path for the following graph

**TRAVELLING SALES PERSON**

1. Describe travelling sales person problem. Find the minimum cost tour for the following graph using the dynamic programming.



2. What is Travelling Sales Man Problem?
3. **Describe travelling sales person problem. Find the minimum cost tour for the following graph using the dynamic programming**



4. Discuss the time and space complexity of dynamic programming of travelling sales person algorithm

RELIABILITY DESIGN

1. Give the statement of Reliability design problem.